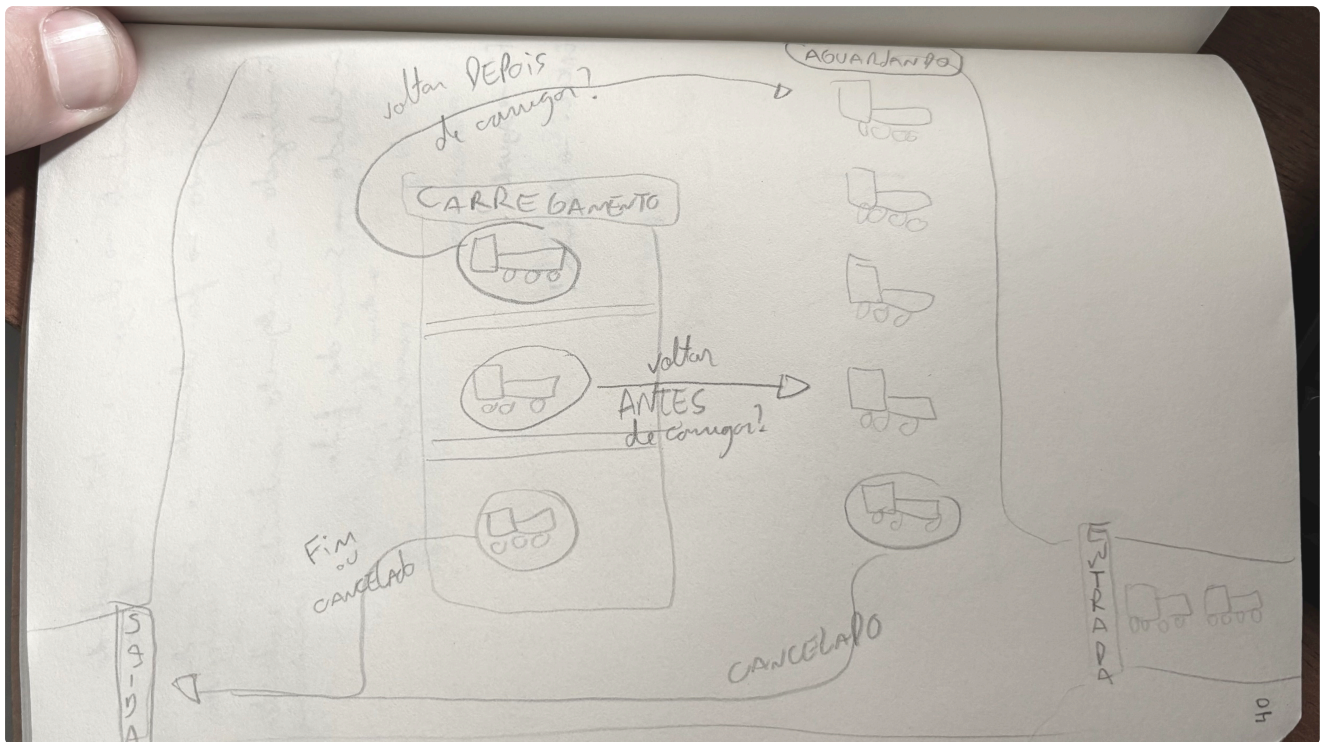


(03/05)

Entendo o problema e desenhando os estados

o problema principal é basicamente o design da "maquina de estados" do projeto (decidir as transições entre estados válidas que modelam o problema de um jeito que faça sentido no mundo real)

motorista chega
fica AGUARDANDO
começa a CARREGAR
termina de carregar
FINALIZA e sai
volta a AGUARDAR
CANCELA e sai
CANCELA e sai



se for possível ir do carregamento de volta para o aguardando, faz diferença se foi antes ou depois de finalizar?

o que cada cenário representa NA PRÁTICA:

- **AGUARDANDO** -> na zona de espera
- **CARREGANDO** -> está numa cabine sendo carregado
- **FINALIZADO** -> saiu da base COM carga
- **CANCELADO** -> saiu da base SEM carga

e o cenário de COM CARGA, porém INCOMPLETA?

ex:

caminhao deveria receber 2 cargas diferentes mas só conseguiu 1
estava carregando e deu algum problema no processo, só carregou metade

CARREGANDO -> AGUARDANDO (antes de terminar)

ex: deu algum problema e o motorista vai esperar de novo

CARREGANDO -> AGUARDANDO (depois de carregar)

ex: caminhao com multiplas cargas (2 carregamentos de combustiveis diferentes?)

nos dois casos, o motorista não sai da base, então ainda não passa por FINALIZADO nem CANCELADO

.... mas daí temos um problema: a entrada na fila tem um campo "produto". Se ele volta pra fila pra carregar *outro* produto, qual produto... como fica?

vejo 2 opções:

- uma entrada por PRODUTO
entrada 1: motorista X, combustivel A -> AGUARDANDO -> CARREGANDO -> FINALIZADO
entrada 2: motorista X, combustivel B -> AGUARDANDO -> CARREGANDO -> FINALIZADO
- uma entrada por VISITA
motorista x, visita A:
AGUARDANDO -> CARREGANDO (combustivel A) -> AGUARDANDO ->
CARREGANDO (combustivel B) -> FINALIZADO

mas se for por VISITA, a modelagem do banco vai ficar mais distante do que foi sugerido no pdf...

1 entrada por PRODUTO fica uma modelagem mais simples e mais coerente com o que é esperado

vou seguir assim por enquanto então:

- FINALIZADO e CANCELADO:
 - são os estados terminais -> não podem mudar para outro estado e toda entrada tem que terminar em algum deles
- AGUARDANDO e CARREGANDO:
 - o caminhão sempre estará em algum desses estados enquanto estiver dentro da base
 - numa mesma visita, o caminhão pode passar por esses estados mais de uma vez
- Para cada carregamento (no caso, para cada produto), haverá 1 entrada

obs: a transição CARREGANDO -> AGUARDANDO representa o caminhão que ainda está na base mas voltou pra esperar (ex: problema no abastecimento, reabastecimento de produto, etc). É UM POUCO DIFERENTE DO CENÁRIO de 2 carregamentos, em que o caminhão também volta da cabine de abastecimento para a zona de espera, mas nesse caso temos 2 registros distintos (FINALIZA o primeiro e começa o segundo como AGUARDANDO).

IDEIA -> coluna "priority" ou algo similar?

Porque tem a fila. Independente se o caminhão foi e voltou para a zona de espera, ele não vai voltar para o fim da fila porque temos a informação do momento de entrada na base. MAS se vamos tratar 2 carregamentos como entradas distintas, não teremos mais essa informação, e não faz sentido o caminhão ter que pegar 1 fila por produto que for carregar. Então ele poderia entrar na fila do segundo produto com prioridade

mas isso eu estou presumindo que precisaria trocar de "cabine" de acordo com produto. Eu introduzi esse problema quando comecei a pensar em múltiplos carregamentos por caminhão. Foi uma suposição minha. Mas isso não faria muito sentido se todos os produtos são carregados no mesmo lugar

à risco de ser preciosista demais, vou pesquisar sobre um base de carregamento pra imaginar melhor o cenário



aparentemente carrega todos os produtos no mesmo lugar mesmo!

...então vamos rever as considerações:

- e o cenário de COM CARGA, porém INCOMPLETA? (ex: caminhão deveria receber 2 cargas diferentes mas só conseguiu 1, estava carregando e deu algum problema no processo, só carregou metade)
 - **INDIFERENTE:** vai sair do "CARREGANDO" para FINALIZADO ou CANCELADO.
- CARREGANDO -> AGUARDANDO (antes de terminar) (ex: deu algum problema e o motorista vai esperar de novo)
 - **PERMANCE UMA SITUAÇÃO VÁLIDA**
- CARREGANDO -> AGUARDANDO (depois de carregar) (ex: caminhão com múltiplas cargas (2 carregamentos de combustíveis diferentes?))
 - O caminhão não precisa sair de uma "cabine de carregamento", ir para a zona de espera, e depois ir pra outra cabine
 - Vai receber todos os carregamentos direto no mesmo lugar
 - VAI FAZER SENTIDO MANTER 2 ENTRADAS NO DB AFINAL?

vejo 2 opções:

- uma entrada por PRODUTO
 entrada 1: motorista X, combustível A -> AGUARDANDO -> CARREGANDO -> FINALIZADO
 entrada 2: motorista X, combustível B -> AGUARDANDO -> CARREGANDO -> FINALIZADO
- uma entrada por VISITA
 motorista x, visita A:
 AGUARDANDO -> CARREGANDO (combustível A) -> AGUARDANDO ->
 CARREGANDO (combustível B) -> FINALIZADO

mas se for por VISITA, a modelagem do banco vai ficar mais distante do que foi sugerido no pdf...

1 entrada por PRODUTO fica uma modelagem mais simples e mais coerente com o que é esperado

IDEIA -> coluna "priority" ou algo similar?

Mantem para casos em que o caminhão teve que ir de CARREGANDO para AGUARDANDO de novo?

(depende... se for manter 1 única entrada, daí não precisa pq a ordem de chamada fica a ordem de chegada)

(04/05)

nova reflexão (e decisão)

será que eu deveria tratar esses edge cases de alguma forma?

Temos isso no pdf: *"Regra obrigatória: Um motorista não pode ter duas entradas ATIVAS (AGUARDANDO ou CARREGANDO) simultaneamente. O backend deve validar e rejeitar*

essa situação com uma mensagem de erro clara."

E isso: "Ponto aberto: Não definimos quais transições de status são permitidas. Por exemplo: um motorista FINALIZADO pode voltar para AGUARDANDO? Um CANCELADO pode ser reativado? Cabe a você definir as regras de negócio e explicá-las no README."

OU SEJA, na verdade se tentarmos criar 2 entradas para múltiplos carregamentos:

Entrada 1: AGUARDANDO <- ativa

Entrada 2: POST /fila <- rejeitado

OU SEJA: múltiplos carregamentos serão tratados de forma sequencial (finaliza primeira entrada, inicia a segunda)

FINALIZADO -> AGUARDANDO: não faz sentido porque o caminhão saiu da base. Se voltar, inicia uma nova visita.

CANCELADO -> reativação: também não faz sentido pq também sai da base e queremos preservar o registro do cancelamento para o histórico. Se o motorista quiser tentar de novo, inicia um nova entrada no db.

CARREGANDO -> AGUARDANDO: Não implementar!! A tabela no db com um "inicio_carregamento" único ficaria inconsistente

Sobre a stack

backend GO

frontend NEXT

banco POSTGRESQL

ja usei NEXT e POSTGRESQL

ainda não usei GO -> vou procurar entender o básico, as diferenças em relação a JS e PHP

instalei no meu linux com

```
sudo apt install golang-go
```

PASTAS

a ideia é ficar algo como

fuel-terminal (raiz)

backend

db - tudo que tem a ver com banco

models - estruturas de dados e regras de negocio

handlers - funcoes que vao responder às ações http

routes -rotas (linkar url->handler)

frontend

types - interfaces do ts

lib - funcoes utilitárias

app - paginas do app (seguindo a estrutura que router do next usa)
components - componentes reutilizaveis que são usados nas pages

BANCO DE DADOS

desenhando o banco

das especificações do projeto:

- Motorista — nome, CPF, CNH, placa do veículo
- Entrada na Fila — referência ao motorista, produto solicitado, status atual, horário de chegada, início e finalização do carregamento
- Produto — nome do combustível (Diesel S10, Biodiesel B100, Gasolina Comum, etc.)

então MOTORISTA e PRODUTO são tabelas....de entidade (pesquisei e descobri que é esse o nome)

e a tabela de ENTRADA_NA_FILA é uma tabela associativa (MAS também tem atributos próprios)

o SQL puro para criar as tabelas ficariam então:

motoristas

```
CREATE TABLE IF NOT EXISTS motoristas (  
  id          SERIAL PRIMARY KEY,  
  nome        VARCHAR(100) NOT NULL,  
  cpf         CHAR(11) UNIQUE NOT NULL,  
  cnh         CHAR(9) UNIQUE NOT NULL,  
  placa       CHAR(7) UNIQUE NOT NULL,  
  created_at  TIMESTAMP DEFAULT NOW()  
);
```

estou usando modelando assim A TITULO DE SIMPLICIDADE, já que o projeto assume

- CPF -> então são motoristas brasileiros (senão teríamos que aceitar documentos mais genericamente)
- CNH -> corrobora a ideia de motoristas brasileiros (caso contrário também teríamos que permitir outros documentos)
- PLACA está atrelada ao motorista -> então presume-se que motorista sempre dirige o mesmo caminhão (nesse projeto não fazemos distinção entre MOTORISTA e VEÍCULO)

Se fossemos fazer esse projeto mais "realisticamente", talvez devêssemos separar MOTORISTA, VEÍCULO e DOCUMENTO? (Criando possíveis múltiplos links entre motoristas e veículos, e também modelando a base para aceitar tipos diferentes de documentos)... mas enfim

Como estou seguindo pelo caminho da simplicidade para esse projeto, adotei os CHARs UNIQUES e NOT NULL considerando que

- CPF tem 11 digitos
- CNH brasileira tem 9 digitos
- Placa brasileira (antiga) e placa mercosul (nova) têm 7 caracteres
- NOT NULL porque todo motorista tem que ter cpf, cnh e uma placa no veículo
- e todos eles são únicos (1 cpf e 1 cnh por motorista, e também 1 placa de acordo com o que falei antes por não considerarmos a separação motorista-veículo)

produtos

a tabela dos produtos é mais simples

```
CREATE TABLE IF NOT EXISTS produtos (  
  id      SERIAL PRIMARY KEY,  
  nome    VARCHAR(100) NOT NULL  
);
```

entradas_fila

```
CREATE TABLE IF NOT EXISTS entradas_fila (  
  id                        SERIAL PRIMARY KEY,  
  motorista_id             INT NOT NULL REFERENCES motoristas(id),  
  produto_id               INT NOT NULL REFERENCES produtos(id),  
  status                   VARCHAR(20) NOT NULL DEFAULT 'AGUARDANDO',  
  horario_chegada           TIMESTAMP NOT NULL DEFAULT NOW(),  
  inicio_carregamento      TIMESTAMP,  
  fim_carregamento         TIMESTAMP  
);
```

basicamente são referencias às outras tabelas, as timestamps, e também o status com default = "AGUARDANDO" (que é o primeiro e único estado possível para o caminhão quando ele acaba de entrar na base)

subindo o banco com Postgre + Docker

por enquanto vou subir direto:

```
docker run --name fuel-terminal-postgres \  
  -e POSTGRES_USER=postgres \  
  -e POSTGRES_PASSWORD=postgres \  
  -e POSTGRES_DB=fuel_terminal \  
  -p 5433:5432 \
```

```
-d \
```

```
postgres:16-alpine
```

```
vecchi@vecchi-linux:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
9df32479857a   postgres:16-alpine   "docker-entrypoint.s..." About a minute ago Up About a minute   0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp   postgres:16-alpine

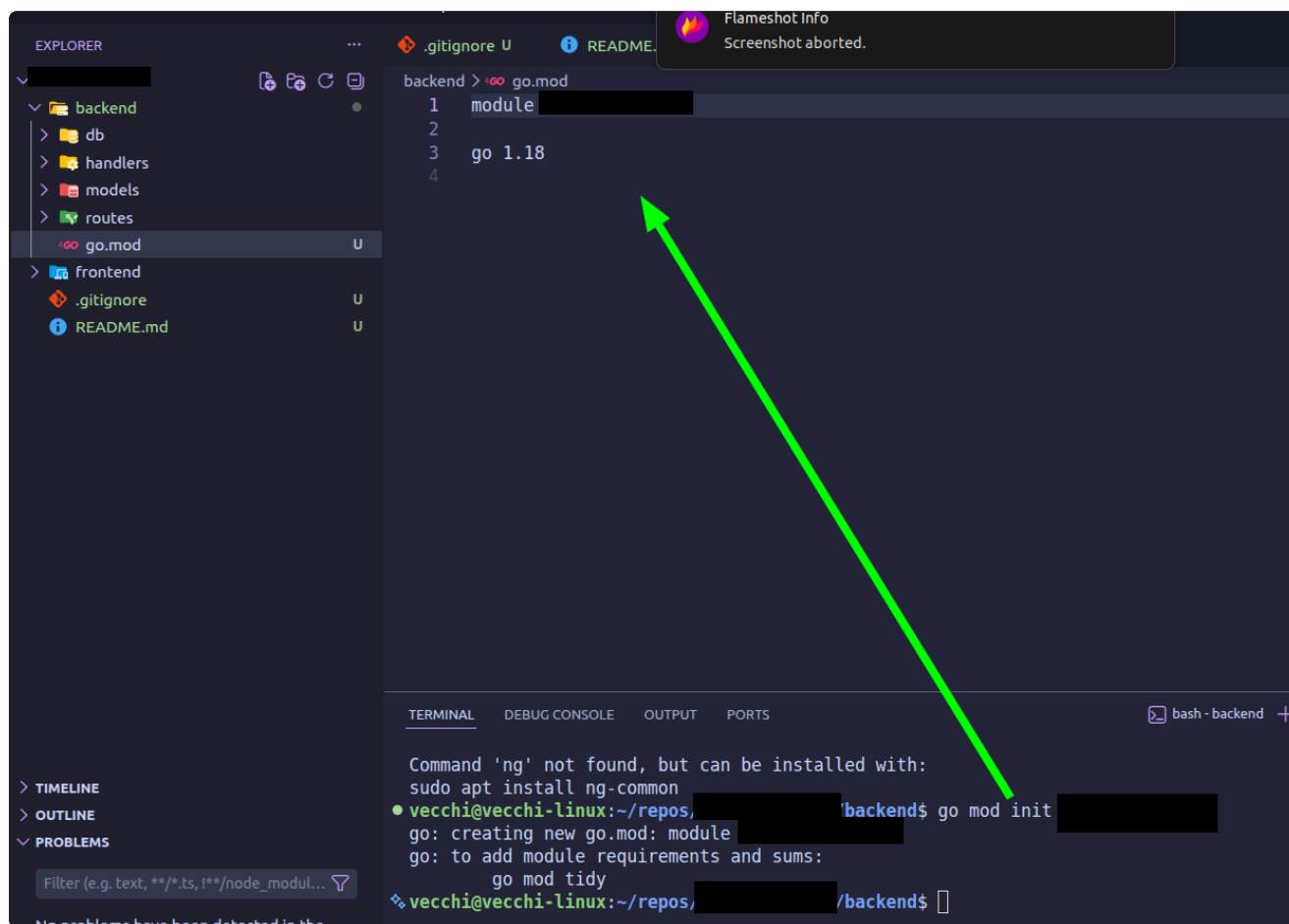
vecchi@vecchi-linux:~$ docker exec -it postgres:16-alpine psql -U postgres -d postgres -c "\l"
List of databases
Name | Owner | Encoding | Locale Provider | Collate | Ctype | ICU Locale | ICU Rules | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----+-----
postgres | postgres | UTF8 | libc | en_US.utf8 | en_US.utf8 |   |   | =c/postgres,+
template0 | postgres | UTF8 | libc | en_US.utf8 | en_US.utf8 |   |   | =c/postgres,+
template1 | postgres | UTF8 | libc | en_US.utf8 | en_US.utf8 |   |   | =c/postgres,+
(4 rows)
```

Está rodando ("up") e o banco "fuel_terminal" aparece normalmente

iniciando módulo Go

pelas minhas pesquisas, vi que o "módulo" do Go é semelhante ao package.json do node, e temos que dar um init nele também lá na pasta do backend com

```
go mod init fuel-terminal
```



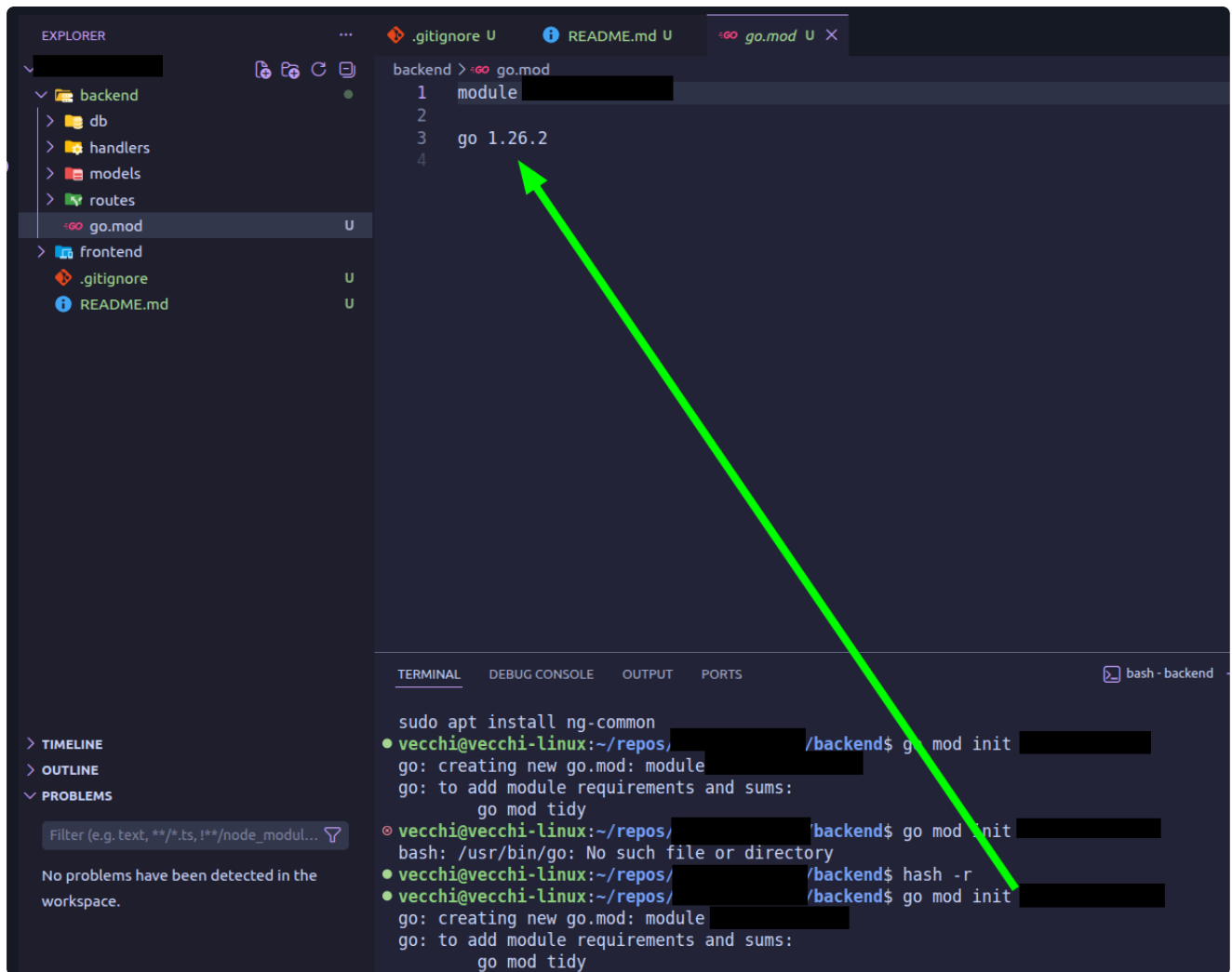
pesquisando também sobre o FIBER e a integração GO->PostgreSQL, descobri que precisamos de 2 dependencias

... vi que o fiber está pedindo Go 1.25+, mas quando instalei veio a versão 1.18...

tive uns probleminhas aqui, mas agora foi -> versão 1.26

```
vecchi@vecchi-linux:~$ go version
go version go1.18.1 linux/amd64
vecchi@vecchi-linux:~$ sudo apt remove --purge golang-go
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  golang-1.18-go golang-1.18-src golang-src
Use 'sudo apt autoremove' to remove them.
The following packages will be REMOVED:
  golang-go*
0 upgraded, 0 newly installed, 1 to remove and 0 not upgraded.
After this operation, 68,6 kB disk space will be freed.
Do you want to continue? [Y/n]
(Reading database ... 365272 files and directories currently installed.)
Removing golang-go:amd64 (2:1.18~0ubuntu2) ...
Processing triggers for man-db (2.10.2-1) ...
vecchi@vecchi-linux:~$ go version
bash: /usr/bin/go: No such file or directory
vecchi@vecchi-linux:~$ which go
/usr/local/go/bin/go
vecchi@vecchi-linux:~$ sudo apt autoremove
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages will be REMOVED:
  golang-1.18-go golang-1.18-src golang-src
0 upgraded, 0 newly installed, 3 to remove and 0 not upgraded.
After this operation, 437 MB disk space will be freed.
Do you want to continue? [Y/n]
(Reading database ... 365244 files and directories currently installed.)
Removing golang-1.18-go (1.18.1-1ubuntu1.2) ...
Removing golang-src (2:1.18~0ubuntu2) ...
Removing golang-1.18-src (1.18.1-1ubuntu1.2) ...
vecchi@vecchi-linux:~$ dpkg -S /usr/bin/go
dpkg-query: no path found matching pattern /usr/bin/go
vecchi@vecchi-linux:~$ which go
type -a go
/usr/local/go/bin/go
go is /usr/local/go/bin/go
vecchi@vecchi-linux:~$ ^[[200~go version~
go: command not found
vecchi@vecchi-linux:~$ go version
bash: /usr/bin/go: No such file or directory
vecchi@vecchi-linux:~$ hash -r
vecchi@vecchi-linux:~$ hash -r
vecchi@vecchi-linux:~$ go version
go version go1.26.2 linux/amd64
```

dei o init de novo e agora tudo certo



instalei o fiber v3 e o driver pq (postgre para go)

```
backend > go.mod
4
Check for upgrades | Upgrade transitive dependencies | Upgrade direct dependencies
5 require (
6     github.com/andybalholm/brotli v1.2.1 // indirect
7     github.com/gofiber/fiber/v3 v3.2.0 // indirect
8     github.com/gofiber/schema v1.7.1 // indirect
9     github.com/gofiber/utls/v2 v2.0.4 // indirect
10    github.com/google/uuid v1.6.0 // indirect
11    github.com/klauspost/compress v1.18.5 // indirect
12    github.com/lib/pq v1.12.3 // indirect
13    github.com/mattn/go-colorable v0.1.14 // indirect
14    github.com/mattn/go-isatty v0.0.21 // indirect
15    github.com/philhofer/fwd v1.2.0 // indirect
16    github.com/tinylib/msgp v1.6.4 // indirect
17    github.com/valyala/bytebufferpool v1.0.0 // indirect
18    github.com/valyala/fasthttp v1.70.0 // indirect
19    golang.org/x/crypto v0.50.0 // indirect
20    golang.org/x/net v0.53.0 // indirect
21    golang.org/x/sys v0.43.0 // indirect
)

TERMINAL  DEBUG CONSOLE  OUTPUT  PORTS

vecchi@vecchi-linux:~/repos/[redacted]/backend$ go get github.com/gofiber/fiber/v3

go: downloading github.com/tinylib/msgp v1.6.4
go: downloading github.com/valyala/bytebufferpool v1.0.0
go: downloading github.com/valyala/fasthttp v1.70.0
go: downloading golang.org/x/net v0.53.0
go: downloading golang.org/x/crypto v0.50.0
go: downloading golang.org/x/sys v0.43.0
go: downloading github.com/philhofer/fwd v1.2.0
go: downloading github.com/andybalholm/brotli v1.2.1
go: downloading github.com/klauspost/compress v1.18.5
go: downloading golang.org/x/text v0.36.0
go: added github.com/andybalholm/brotli v1.2.1
go: added github.com/gofiber/fiber/v3 v3.2.0
go: added github.com/gofiber/schema v1.7.1
go: added github.com/gofiber/utls/v2 v2.0.4
go: added github.com/google/uuid v1.6.0
go: added github.com/klauspost/compress v1.18.5
go: added github.com/mattn/go-colorable v0.1.14
go: added github.com/mattn/go-isatty v0.0.21
go: added github.com/philhofer/fwd v1.2.0
go: added github.com/tinylib/msgp v1.6.4
go: added github.com/valyala/bytebufferpool v1.0.0
go: added github.com/valyala/fasthttp v1.70.0
go: added golang.org/x/crypto v0.50.0
go: added golang.org/x/net v0.53.0
go: added golang.org/x/sys v0.43.0
go: added golang.org/x/text v0.36.0
● vecchi@vecchi-linux:~/repos/[redacted]/backend$ go get github.com/lib/pq
go: downloading github.com/lib/pq v1.12.3
go: added github.com/lib/pq v1.12.3
❖ vecchi@vecchi-linux:~/repos/[redacted]/backend$
```

(05/05) parte 1 - 5h46

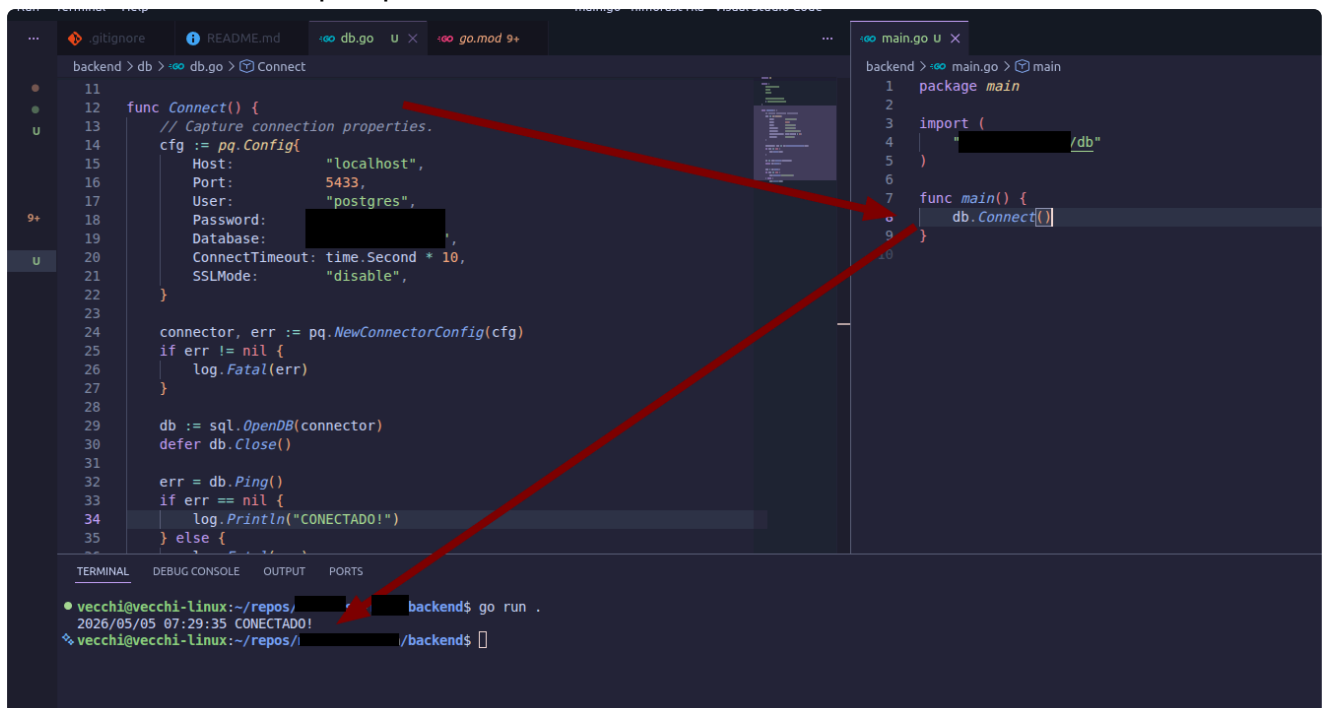
Conectando projeto com o db

estou seguindo <https://go.dev/doc/tutorial/database-access> e <https://github.com/lib/pq> para entender como conectar com o banco em postgresql

também estou pesquisando sobre como usar variáveis de ambiente em Go, de maneira similar ao Node com .env

vou criar o backend/db/db.go

depois de um tempo batendo cabeça aqui e lendo documentações, consegui conectar ao banco e entender um pouquinho mais sobre Go



```
backend > db > db.go > Connect
11
12 func Connect() {
13     // Capture connection properties.
14     cfg := pq.Config{
15         Host:     "localhost",
16         Port:     5433,
17         User:     "postgres",
18         Password: " ",
19         Database: " ",
20         ConnectTimeout: time.Second * 10,
21         SSLMode:   "disable",
22     }
23
24     connector, err := pq.NewConnectorConfig(cfg)
25     if err != nil {
26         log.Fatal(err)
27     }
28
29     db := sql.OpenDB(connector)
30     defer db.Close()
31
32     err = db.Ping()
33     if err == nil {
34         log.Println("CONECTADO!")
35     } else {
36
37     }
38 }
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

backend > main.go > main
1 package main
2
3 import (
4     " " /db"
5 )
6
7 func main() {
8     db.Connect()
9 }
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

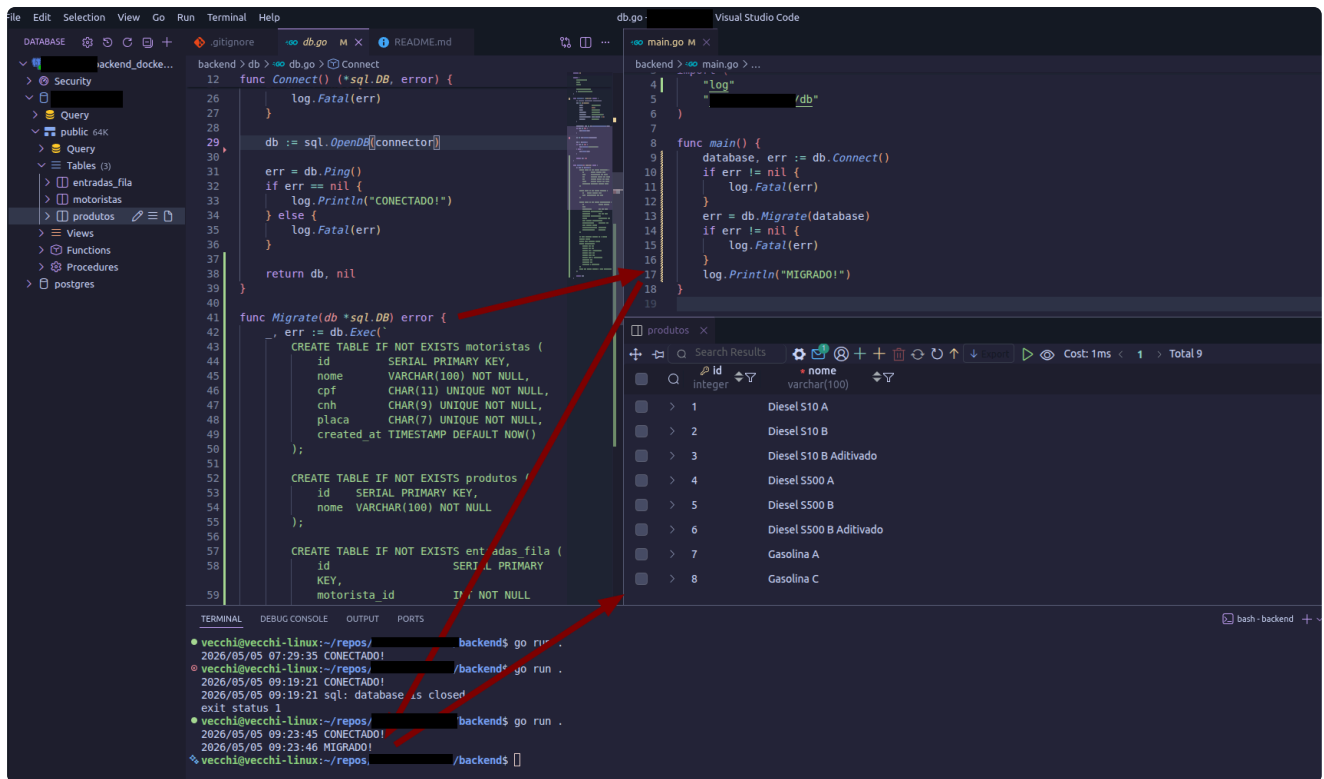
TERMINAL DEBUG CONSOLE OUTPUT PORTS
● vecchi@vecchi-linux:~/repos/ backend$ go run .
2026/05/05 07:29:35 CONECTADO!
❖ vecchi@vecchi-linux:~/repos/ /backend$
```

criar tabelas no banco via código (configurar uma função "Migrate")

NOTA: os produtos são

- Diesel S10 A
- Diesel S10 B
- Diesel S10 B Aditivado
- Diesel S500 A
- Diesel S500 B
- Diesel S500 B Aditivado
- Gasolina A
- Gasolina C
- Gasolina C Aditivada

usei as queries que tinha desenhado nesse documento e criei uma função Migrate -> funcionou: as tabelas foram criadas no db e os valores do seed foram preenchidos como esperado



(OBS: descobri essa sintaxe "_,err" e "db.Exec" aqui: <https://www.youtube.com/watch?v=Y7a0sNKdoQk>)

PARA LEMBRAR: esse "_,err" é porque vamos descartar o primeiro valor que o db.Exec() retorna (no caso, retorna "result sql.Result, err error"), já que não nos interessa

(05/05) parte 2 - 16h50

models

criando o models.go para definir os modelos de dados do app

OBS: estou tentando manter tudo o que tiver a ver com o negócio em português. Já passei casos em que inventamos de escrever tudo em inglês e tem hora que não traduz bem, ou certas coisas podem ficar ambíguas (ex: na empresa que trabalho com educação, "grade = 8" -> pode ser "8th grade" de "8o ano" ou "nota 8")

preciso criar os types que representam as tabelas do db (motorista, produto, entrada_na_fila)

o que faria em TS como:

```
type Motorista = { id: number; nome: string; cpf: string; ... }
```

aprendi que nesse caso eu utilizo STRUCTS

e que posso definir como os campos aparecem no json pra integrar com o front depois, por exemplo

e que "*" define o que é opcional

e que "omitempty"...omite quando empty (se é nil, não inclui no json)

algumas decisões na modelagem dos structs

"Motorista" e "Produto" são mais simples. Tirando o id, tudo é string e tem que estar preenchido

agora, o "EntradaFila" estava pensando aqui:

ia deixar "motorista" e "produto" obrigatórios, porque a tabela "entrada_na_fila" precisa dos ids deles como obrigatório, mas daí sempre será necessário conversar com as outras tabela pra qualquer coisa (ou seja, sempre vai precisar dos JOINS)

- achei melhor separar "Motorista" e "MotoristaID", porque daí posso deixar só o ID como obrigatório, e os dados completos só passo quando for necessário (mesma coisa para produto)

Status não fica legal usar um "string"...

acredito que merece um type Status para evitar typos por ficar reescrevendo os estados toda vez e deixar a maquina de estados mais segura (fora que se for mudar alguma coisa, fica centralizado)

```
type Status string

const (
    StatusAguardando Status = "AGUARDANDO"
    StatusCarregando  Status = "CARREGANDO"
    StatusFinalizado  Status = "FINALIZADO"
    StatusCancelado   Status = "CANCELADO"
)
```



descobri isso daqui (methods on types -> parece metodos de classe)

<https://go.dev/tour/methods/1>

Methods

Go does not have classes. However, you can define methods on types.

A method is a function with a special *receiver* argument.

The receiver appears in its own argument list between the `func` keyword and the method name.

In this example, the `Abs` method has a receiver of type `Vertex` named `v`.

```
1 package main
2
3 import (
4     "fmt"
5     "math"
6 )
7
8 type Vertex struct {
9     X, Y float64
10 }
11
12 func (v Vertex) Abs() float64 {
13     return math.Sqrt(v.X*v.X + v.Y*v.Y)
14 }
15
16 func main() {
17     v := Vertex{3, 4}
18     fmt.Println(v.Abs())
19 }
20
```

Re

voltando ao EntradaFila: inicio e FimCarregamento são opcionais
então meu models.go vai ficar assim por enquanto:

```
package models

import (
    "time"
)

type Status string

const (
    StatusAguardando Status = "AGUARDANDO"
    StatusCarregando  Status = "CARREGANDO"
    StatusFinalizado  Status = "FINALIZADO"
    StatusCancelado   Status = "CANCELADO"
)

type Motorista struct {
    ID      int    `json:"id"`
    Nome    string  `json:"nome"`
    CPF     string  `json:"cpf"`
    CNH     string  `json:"cnh"`
    Placa   string  `json:"placa"`
}

type Produto struct {
    ID      int    `json:"id"`
    Nome    string  `json:"nome"`
}
```

```

type EntradaFila struct {
    ID                int        `json:"id"`
    MotoristaID       int        `json:"motorista_id"`
    Motorista         *Motorista `json:"motorista,omitempty"`
    ProdutoID         int        `json:"produto_id"`
    Produto           *Produto  `json:"produto,omitempty"`
    Status            Status     `json:"status"`
    HorárioChegada    time.Time `json:"horario_chegada"`
    InicioCarregamento *time.Time `json:"inicio_carregamento,omitempty"`
    FimCarregamento  *time.Time `json:"fim_carregamento,omitempty"`
}

```

(06/05 - 5h51)

pensando em api/rotas, handlers..

Método	Rota	Descrição	Auth
POST	/fila	Registra a chegada de um motorista na fila	■
GET	/fila	Lista todos os motoristas na fila do dia, ordenados por chegada	■
PATCH	/fila/:id/status	Atualiza o status de uma entrada na fila	■
GET	/fila/:id	Retorna os detalhes de uma entrada específica	■
GET	/fila/historico	Lista entradas finalizadas e canceladas (dias anteriores)	■
GET	/motoristas/busca	Busca motorista por CPF ou placa para pré-preenchimento do formulário	■
GET	/produtos	Lista os produtos disponíveis para carregamento	■

vou precisar dar uma olhada no fiber

<https://docs.gofiber.io/#hello-world>

<https://docs.gofiber.io/guide/routing#paths>

OBS: para daqui a pouco - CORS - <https://docs.gofiber.io/middleware/cors>

...bom, vamos começar com um teste basico HelloWorld + rotas simples para ver em funcionamento

The image shows a development environment with three main components: a file explorer, a code editor, and a terminal.

File Explorer: The left sidebar shows a project structure with folders like `backend`, `db`, `handlers`, `models`, `routes`, `go.mod`, `go.sum`, `main.go` (selected), `frontend`, `.gitignore`, `README.md`, and `test.js`.

Code Editor: The main area displays the `main.go` file. The code defines a package `main` and imports `log` and `github.com/gofiber/fiber/v3`. It includes a `main` function that sets up a Fiber app, connects to a database, and listens on port 3000. The code is as follows:

```
1 package main
2
3 import (
4     "log"
5     "github.com/gofiber/fiber/v3"
6 )
7
8 // func main() {
9 //     database, err := db.Connect()
10 //     if err != nil {
11 //         log.Fatal(err)
12 //     }
13 //     err = db.Migrate(database)
14 //     if err != nil {
15 //         log.Fatal(err)
16 //     }
17 //     log.Println("MIGRADO!")
18 // }
19
20 func main() {
21     app := fiber.New()
22     app.Get("/", func(c fiber.Ctx) error {
23         return c.SendString("Hello, World!")
24     })
25     app.Listen(":3000")
26 }
```

Terminal: The bottom panel shows the command prompt. The user has run `go run .` in the `backend` directory. The output shows a warning about a redeclared `main` function and a successful execution of the program.

Browser: A browser window is open at `localhost:3000`. The page displays an error message: "This site can't be reached. localhost refused to connect." Two pink arrows point from the browser window to the code editor: one points to the `app.Listen(":3000")` line, and the other points to the `go run .` command in the terminal.

ah, não tinha salvado as mudanças 🤖

The screenshot shows a VS Code editor with a Go file named `main.go` open. The code defines a Fiber web application that listens on port 3000 and responds with "Hello, World!". The terminal at the bottom shows the command `go run .` being executed, which results in a "main redeclared in this block" error. A web browser window is also open, displaying "Hello, World!" at `localhost:3000`.

```
backend > go main.go > ...
4 | "github.com/gofiber/fiber/v3"
5 | )
6 |
7 | // func main() {
8 | //   database, err := db.Connect()
9 | //   if err != nil {
10 | //     log.Fatal(err)
11 | //   }
12 | //   err = db.Migrate(database)
13 | //   if err != nil {
14 | //     log.Fatal(err)
15 | //   }
16 | //   log.Println("MIGRADO!")
17 | // }
18 |
19 | func main() {
20 |   app := fiber.New()
21 |
22 |   app.Get("/", func(c fiber.Ctx) error {
23 |     return c.SendString("Hello, World!")
24 |   })
25 |
26 |   app.Listen(":3000")
27 | }
28 |
```

TERMINAL

```
Command 'ng' not found, but can be installed with:
sudo apt install ng-common
● vecchi@vecchi-linux:~/repos/...$ cd backend/
○ vecchi@vecchi-linux:~/repos/.../backend$ go run .
#
./main.go:22:6: main redeclared in this block
./main.go:10:6: other declaration of main
❖ vecchi@vecchi-linux:~/repos/.../backend$ go run .
```

workspace.

v3.2.0

```
INFO Server started on: http://127.0.0.1:3000 (bound on host 0.0.0.0 and port 3000)
INFO Total handlers: 2
INFO Prefork: Disabled
INFO PID: 11217
INFO Total process count: 1
```

certo. Agora vamos montar algumas rotas de teste

como faço o equivalente a "--watch" para atualizar o server automaticamente quando salvar mudanças?

fiber CLI -> tem o comando "fiber dev" que oferece live reload

mas nao consigo rodar depois de instalar...

depois de brigar com o PATH do go no linux, consegui rodar

The screenshot shows a VS Code editor with a Go Fiber application in `main.go`. The code defines a simple web server with two routes: a root route returning "Hello, World!" and a `/teste` route returning "rota de teste". The application is started using `fiber dev` in the terminal. The terminal output shows the compilation and server startup on port 3000. A browser window at `localhost:3000/teste` displays the text "rota de teste".

```
backend > main.go > main
// func main() {
//     // database, err := db.Connect()
//     // if err != nil {
//     //     log.Fatal(err)
//     // }
//     // err = db.Migrate(database)
//     // if err != nil {
//     //     log.Fatal(err)
//     // }
//     log.Println("MIGRADO!")
// }

func main() {
    app := fiber.New()

    app.Get("/", func(c fiber.Ctx) error {
        return c.SendString("Hello, World!")
    })

    app.Get("/teste", func(c fiber.Ctx) error {
        return c.SendString("rota de teste")
    })

    app.Listen(":3000")
}
```

```
vecchi@vecchi-linux:~/repos/.../backend$ which fiber
/home/vecchi/go/bin/fiber
vecchi@vecchi-linux:~/repos/.../backend$ fiber dev
2026/05/06 06:38:29 Welcome to fiber dev
2026/05/06 06:38:29 Compiling...
2026/05/06 06:38:29 Add /home/vecchi/repos/.../backend to watch
2026/05/06 06:38:29 Add /home/vecchi/repos/.../backend/db to watch
2026/05/06 06:38:29 Add /home/vecchi/repos/.../backend/handler to watch
2026/05/06 06:38:29 Add /home/vecchi/repos/.../backend/models to watch
2026/05/06 06:38:29 Add /home/vecchi/repos/.../backend/routes to watch
2026/05/06 06:38:29 Compile done in 604.12ms!
2026/05/06 06:38:29 New pid is 15245
```

INFO Server started on: http://127.0.0.1:3000 (bound on host 0.0.0.0 and port 3000)
INFO Total handlers: 4
INFO Prefork: Disabled
INFO PID: 15245
INFO Total process count: 1

ok, funciona igual o Express (ordem das rotas é importante)

The screenshot shows a VS Code editor with a Go Fiber application in `main.go`. The code defines three routes: a root route returning "Hello, World!", a parameterized route `/:teste` returning "teste parametro: " followed by the parameter value, and a `/teste` route returning "rota de teste". The application is started using `fiber dev`. A browser window at `localhost:3000/teste` displays the text "teste parametro: teste", with a pink arrow pointing from the `/:teste` route in the code to the browser output.

```
backend > main.go > main
// func main() {
//     // database, err := db.Connect()
//     // if err != nil {
//     //     log.Fatal(err)
//     // }
//     // err = db.Migrate(database)
//     // if err != nil {
//     //     log.Fatal(err)
//     // }
//     log.Println("MIGRADO!")
// }

func main() {
    app := fiber.New()

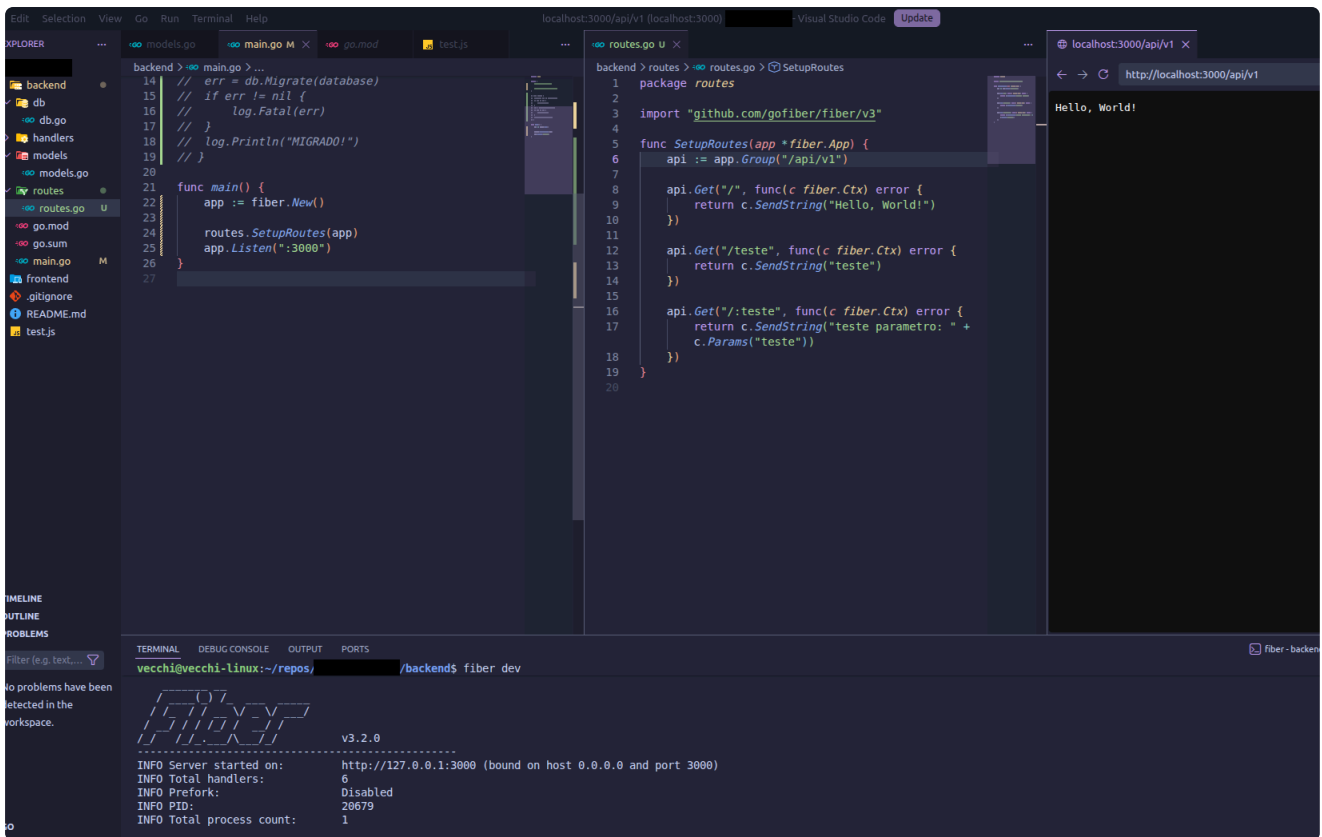
    app.Get("/", func(c fiber.Ctx) error {
        return c.SendString("Hello, World!")
    })

    app.Get("/:teste", func(c fiber.Ctx) error {
        return c.SendString("teste parametro: " + c.Params("teste"))
    })

    app.Get("/teste", func(c fiber.Ctx) error {
        return c.SendString("rota de teste")
    })

    app.Listen(":3000")
}
```

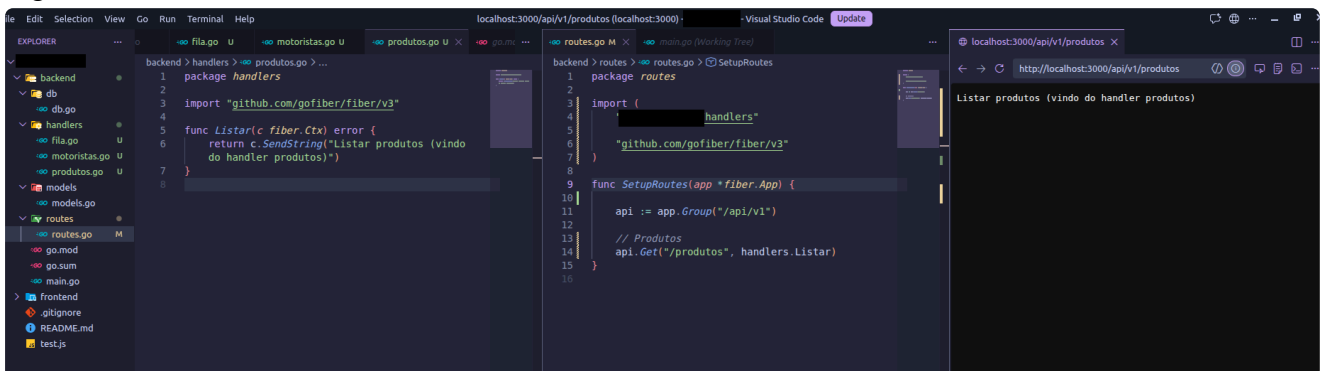
teste passando as rotas para um arquivo "routes" e agrupando as rotas em `/api/v1` (como solicitado no briefing do projeto)



volto logo..

TO-DO: criar todos os endpoints e separar os handlers (ainda não escrever as funções, mas deixar o esqueleto montado)

legal, funciona



mas desse jeito, todo mundo vai ficar junto em "handlers" e os métodos vão ficar confusos (ex: "Listar" não fica claro "listar o que" - ao mesmo tempo não quero ter que ficar escrevendo "ListarProdutos" "ListarMotoristas" se estarão em arquivos separados... teoricamente cada um poderia ter seu "Listar" separado)

OU SEJA: preciso fazer essa distinção a nível de handler, não método

.. nesse caso cada handler é um package diferente?...

nao, pq todos eles fazem parte do package "handlers", daí é só chamar de alguma forma "handlers.produtos" e depois "handlers.produtos.Listar"

então é como se fosse.. uma classe com seus métodos

então posso fazer fazer o type, já que posso definir métodos específicos para um type

[^673c76](#)

mas não pode ser um "type Produto" porque isso é o que usamos no model...
esses métodos são específicos do handler de produtos, no caso
então "type ProdutoHandler"?

assim:

```
backend > handlers > produtos.go > Produto
1 package handlers
2
3 import "github.com/gofiber/fiber/v3"
4
5 type ProdutoHandler struct {
6     // .... mas não sei o que colocar aqui :,)
7 }
8
9 func Listar(c fiber.Ctx) error {
10     return c.SendString("Listar produtos (vindo do handler produtos)")
11 }
12
```

mas aí fica "vazio" só pra termos os métodos "agrupados em algum lugar"?

bom, seguimos assim por enquanto

mas não posso só fazer isso:

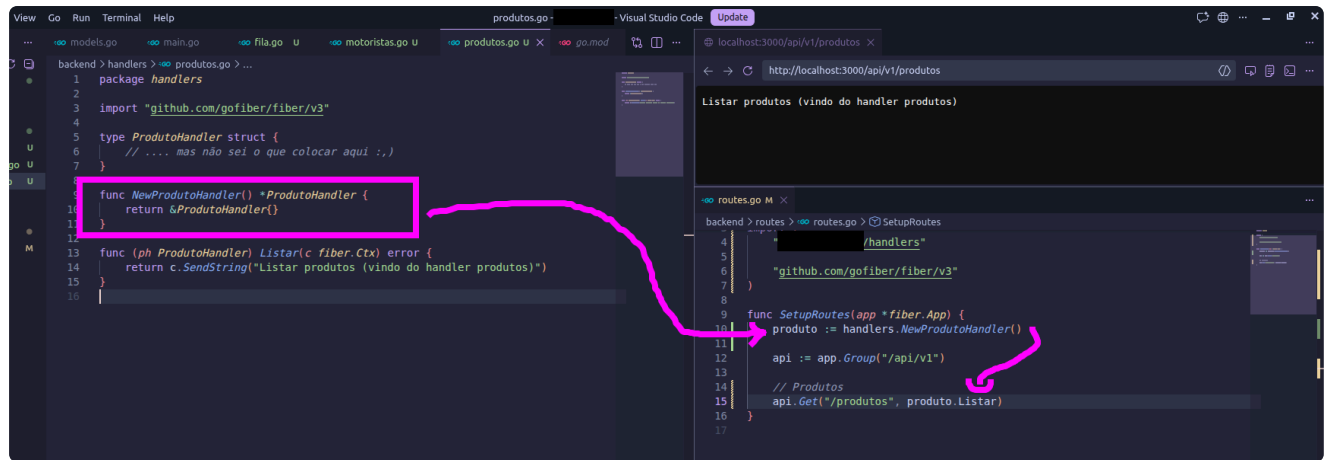
```
backend > handlers > produtos.go > ...
1 package handlers
2
3 import "github.com/gofiber/fiber/v3"
4
5 type ProdutoHandler struct {
6     // .... mas não sei o que colocar aqui :,)
7 }
8
9 func (ph ProdutoHandler) Listar(c fiber.Ctx) error {
10     return c.SendString("Listar produtos (vindo do handler produtos)")
11 }
12

backend > routes > routes.go > ...
1 package routes
2
3 import (
4     "github.com/gofiber/fiber/v3"
5     "github.com/gofiber/fiber/v3/middleware/logger"
6     handlers "github.com/gofiber/fiber/v3/handlers"
7 )
8
9 func SetupRoute(app *fiber.App) {
10     produto := handlers.ProdutoHandler
11     api := app.Group("/api/v1")
12     // Produtos
13     api.Get("/produtos", handlers.Listar)
14 }
15
16
```

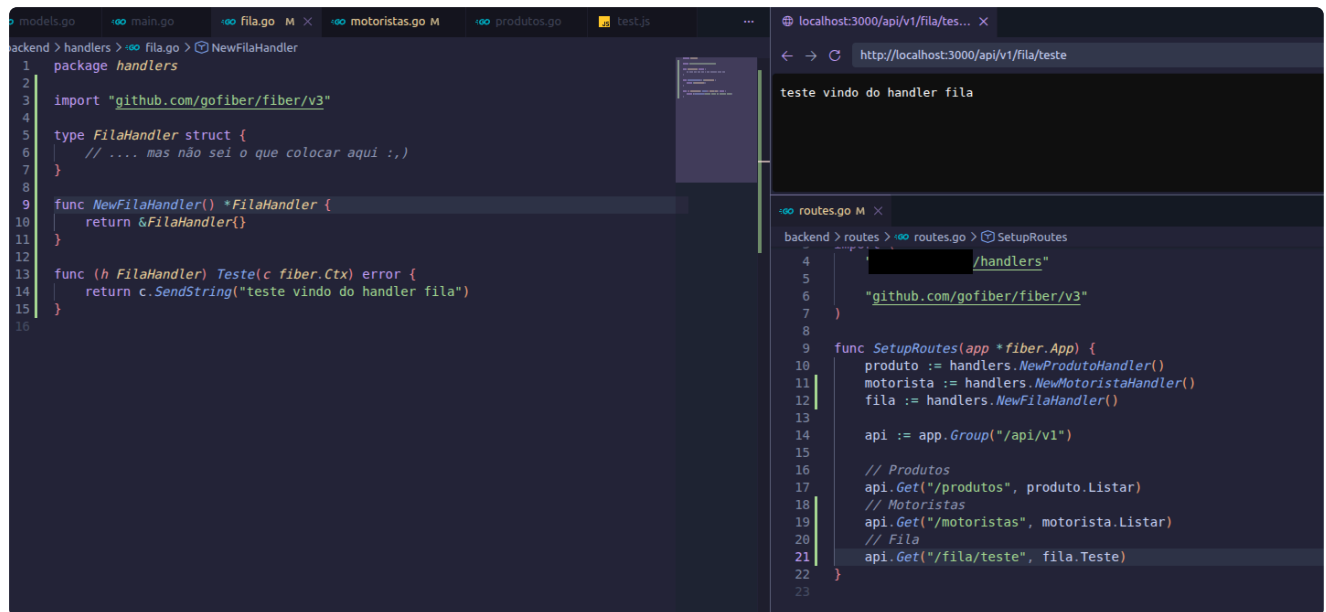
É NÉ. É porque assim eu estou falando do TYPE, mas eu preciso de uma INSTANCE
... como eu faço como "new ProdutoHandler()"?

Porque ProdutoHandler seria o modelo, agora eu preciso de um ProdutoHandler "concreto"

descobri como instanciar direto e também como fazer um constructor (que é como seguirei
pra deixar mais organizado, assim deixo o constructor de cada um no seu arquivo)



caminhando



agora vamos montar de fato todos os endpoints (só sem implementar as funções de fato ainda)

The image shows a VS Code editor with a Go project. The main editor displays the `handlers.go` file, which defines several handlers for a web application. A pink box highlights the `CriarEntrada` and `AtualizarStatus` functions. A pink arrow points from the terminal to the `AtualizarStatus` function. The right sidebar shows the `routes.go` file, which defines the routes for the application. The bottom terminal shows the output of several curl commands, including a PATCH request to update a status and a POST request to create a new entry.

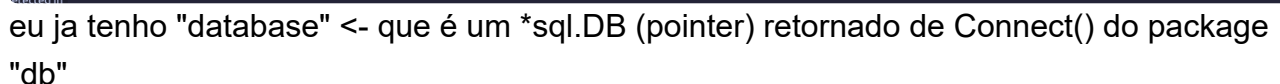
```
backend > handlers > fila.go > ...
4
5 type FilaHandler struct {
6     // .... mas não sei o que colocar aqui :,)
7 }
8
9 func NewFilaHandler() *FilaHandler {
10     return &FilaHandler{}
11 }
12
13 func (h FilaHandler) CriarEntrada(c fiber.Ctx) error {
14     return c.SendString("Registra a chegada de um motorista na fila ")
15 }
16
17 func (h FilaHandler) AtualizarStatus(c fiber.Ctx) error {
18     return c.SendString("Atualiza o status de uma entrada na fila")
19 }
20
21 func (h FilaHandler) ListarEntradas(c fiber.Ctx) error {
22     return c.SendString("Lista todos os motoristas na fila do dia, ordenados
23     por chegada")
24 }
25
26 func (h FilaHandler) BuscarEntrada(c fiber.Ctx) error {
27     return c.SendString("Retorna os detalhes de uma entrada especifica")
28 }
29
30 func (h FilaHandler) Historico(c fiber.Ctx) error {
31     return c.SendString("Lista entradas finalizadas e canceladas (dias
32     anteriores)")
33 }
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
backend > routes > routes.go > SetupRoutes
9 func SetupRoutes(app *fiber.App) {
10     api.Get("/motoristas/busca", motorista.Buscar)
11 }
12
13 // Fila
14 api.Get("/fila/historico", fila.Historico)
15 api.Get("/fila/:id", fila.BuscarEntrada)
16 api.Get("/fila", fila.ListarEntradas)
17 api.Patch("/fila/:id/status", fila.AtualizarStatus)
18 api.Post("/fila", fila.CriarEntrada)
19 }
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
vecchi@vecchi-linux:~/repos $ curl -X PATCH http://localhost:3000/api/v1/fila/123/status
{"status": "cancelada"}
vecchi@vecchi-linux:~/repos $ curl -X POST http://localhost:3000/api/v1/fila
{"id": "123", "status": "cancelada", "data": "2023-01-01T00:00:00Z"}
vecchi@vecchi-linux:~/repos $
```

a api já funciona, agora falta devolver os dados que queremos para cada rota -> falta conversar com o banco

https://go.dev/doc/database/querying#single_row - Aqui achei alguns métodos do "DB"



```
// JS always uses references (pointers), so this is automatic:
let x = { name: "handler" };
let y = x;
y.name = "modified"; // x.name is now "modified" too!
// They point to the SAME object in memory

// Go WITHOUT & (copies):
x := ProdutoHandler{name: "handler"}
y := x
y.name = "modified" // x.name is still "handler" (different copy)

// Go WITH & (references, like JS):
x := ProdutoHandler{name: "handler"}
y := &x
// Now y points to x. They're the same thing in memory.
```

```
x := 10
ptr := &x // ptr now holds the memory address of x
fmt.Println(ptr) // Output: 0xc000... (a hexadecimal address)
fmt.Println(*ptr) // Output: 10 (the value stored at that address)
```

então

```
// database *sql.DB -> eu recebo um pointer para o db que já existe (não uma cópia)
// *ProdutoHandler -> e retorno um pointer para uma instância de type
// ProdutoHandler (não uma cópia)
func NewProdutoHandler(database *sql.DB) *ProdutoHandler {
    return &ProdutoHandler{}
    // e aqui eu CRIO UMA INSTÂNCIA do tipo ProdutoHandler e RETORNO UMA
    // REFERENCE a essa instância
}
```

(07/05)

conversei com o Claude para conferir se meu entendimento estava correto:

 ****resposta do claude:****

- `*sql.DB` — yes, exactly. The database connection is a heavy object with connection pools and state. You want one shared instance, not copies flying around.
- `ProdutoHandler` struct — yes, same as a TypeScript type/interface. It's a blueprint for creating similar entities.
- `*ProdutoHandler` as return type — yes, you're promising to return a pointer to one.

- `&ProdutoHandler{}` — yes, you create the struct right there, Go allocates it in memory, and `&` gives you its address. Exactly like `new ProdutoHandler()` in JS, except Go gives you the address explicitly.

Why return a pointer instead of the value?

A few reasons that compound each other:

1. Mutations stay consistent. If you later add methods that modify the struct (like caching something), a pointer means everyone holding a reference to this handler sees the same state. A copy would diverge silently.

2. The methods are defined on `*ProdutoHandler`. Notice your method:

go

```
func (h ProdutoHandler) Listar(c fiber.Ctx) error { ... }
```

This works either way right now, but in Go it's a convention: if *any* method on a type uses a pointer receiver `(h *ProdutoHandler)`, all methods should, and you should pass pointers around. It's easier to stay consistent from the start.

3. It signals "this is a shared thing, not a throwaway value." In Go, returning a value means "here's your own copy, do what you want." Returning a pointer means "here's *the* handler." For something wired up in your router, the pointer intent is clearer.

The JS analogy that might click:

In JavaScript, objects are *always* passed by reference — you never think about it. In Go, structs are passed by value by default (like if JS primitives were objects). The `*` and `&` are just Go making you *explicit* about what JS did silently for you.

go

```
// This is what Go does implicitly in JS for every object:  
handler := &ProdutoHandler{} // "give me the address of this thing"  
produto := *handler           // "give me a copy of the thing at this  
address"
```

Your instincts are solid — you're reading the code correctly.

eu estava encanado porque tinha a suspeita que NO CENÁRIO ESPECÍFICO DO MEU CÓDIGO ATÉ ESSE MOMENTO, não teria problema se eu retornasse o struct de fato (não reference), e ele confirmou que funcionaria mesmo. Mas eu tinha lido que era uma boa prática, uma convenção, retornar a reference e agora ficou mais claro o porque.

SEGUIMOS.

seguindo essa mesma ideia, todo Handler precisa ter acesso ao db (no caso, pointer para que todo mundo aponte para a mesma instância), então vamos incluir isso na declaração dos types de cada Handler

OBS:

```
type ProdutoHandler struct {
    db *sql.DB
}

// É DIFERENTE DE:

type ProdutoHandler struct {
    db: *sql.DB
    // fiz assim no automático pelo costume do TS, mas a sintaxe do Go é a
    de cima
}
```

também precisamos alterar os constructors

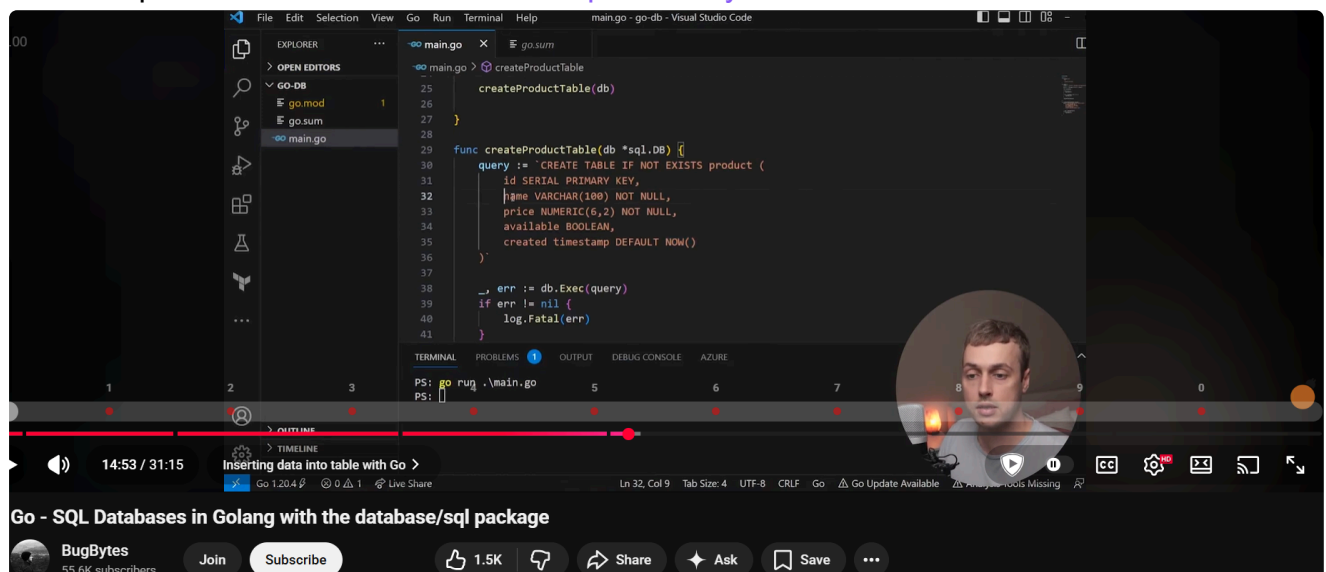
```
func NewFilaHandler(database *sql.DB) *FilaHandler {
    return &FilaHandler{db: database}
}

// AQUI SIM usamos o ":" para definir o valor de um atributo
```

agora preciso checar como fazer uma função do handler interagir de fato com o banco

vou começar pelo "Listar()" dos produtos, já que o banco já tem dados nessa tabela

voltando pro video sobre dbs de novo <https://www.youtube.com/watch?v=Y7a0sNKdoQk>



Key Differences

- **Query()** : Used for SQL statements that **return rows** (`SELECT`). It returns `*sql.Rows` , which you iterate over using `.Next()` and `.Scan()` .
- **Exec()** : Used for SQL statements that **do not return rows** (`INSERT` , `UPDATE` , `DELETE` , `CREATE`). It returns a `sql.Result` containing information about rows affected or the last inserted ID.  The Go Programming Language +4

```
9 type ProdutoHandler struct {
10     db *sql.DB
11 }
12
13 func NewProdutoHandler(db *sql.DB) *ProdutoHandler {
14     return &ProdutoHandler{db: db}
15 }
16
17 func (h *ProdutoHandler) ListarProdutos() {
18     sql := "SELECT id, nome, preco FROM produtos"
19     rows, err := h.db.Query(sql)
20     if err != nil {
21         log.Fatal(err)
22     }
23     defer rows.Close()
24     return c.SendString("Lista os produtos disponíveis para carregamento")
25 }
```

func (db *sql.DB) Query(query string, args ...any) (*sql.Rows, error)

Query executes a query that returns rows, typically a SELECT. The args are for any placeholder parameters in the query.

Query uses context.Background internally; to specify the context, use DB.QueryContext.

(sql.DB).Query on pkg.go.dev

ok... Query() executa a query "sql" e retorna um *sql.Rows
.... e daí?

https://go.dev/doc/database/querying#multiple_rows

ESSE É O PADRÃO

```
func albumsByArtist(artist string) ([]Album, error) {
    rows, err := db.Query("SELECT * FROM album WHERE artist = ?", artist)
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    // An album slice to hold data from returned rows.
    var albums []Album

    // Loop through rows, using Scan to assign column data to struct fields.
    for rows.Next() {
        var alb Album
        if err := rows.Scan(&alb.ID, &alb.Title, &alb.Artist,
            &alb.Price, &alb.Quantity); err != nil {
            return albums, err
        }
        albums = append(albums, alb)
    }
    if err = rows.Err(); err != nil {
        return albums, err
    }
    return albums, nil
}
```

- `defer rows.Close()` -> "to free connections"

- for rows.Next() -> "to iterate over rows"
- rows.Err() -> "check for errors encountered during iteration"

sei como iterar, mas agora preciso entender como montar isso.... seria um map? Um slice?

```
func (h ProdutoHandler) Listar(c fiber.Ctx) error {
    sql := "SELECT id, nome FROM produtos"
    rows, err := h.db.Query(sql)
    if err != nil {
        log.Fatal(err)
    }
    defer rows.Close()

    for rows.Next() {
        // escanear cada row e um "objeto" que seja uma lista de
        // produtos
    }

    // retornar JSON com a lista
}
```

vamos dar uma checada na documentação

NAO RELACIONADO MAS INTERESSANTE:

Exported names

In Go, a name is exported if it begins with a capital letter. For example, `Pizza` is an exported name, as is `Pi`, which is exported from the `math` package.

estudando sobre arrays, slices, maps....

para quando voltar: <https://www.youtube.com/watch?v=v3RodjH1i6c&list=PL4cUxeGkcC9gC88BEo9czgyS72A3doDeM&index=12>

(08/05)

https://go.dev/doc/effective_go

Allocation with make ¶

Back to allocation. The built-in function `make(T, args)` serves a purpose different from `new(T)`. It creates slices, maps, and channels only, and it returns an *initialized (not zeroed) value of type T (not *T)*. The reason for the distinction is that these three types represent, under the covers, references to data structures that must be initialized before use. A slice, for example, is a three-item descriptor containing a pointer to the data (inside an array), the length, and the capacity, and until those items are initialized, the slice is `nil`. For slices, maps, and channels, `make` initializes the internal data structure and prepares the value for use. For instance,

```
make([]int, 10, 100)
```

allocates an array of 100 ints and then creates a slice structure with length 10 and a capacity of 100 pointing at the first 10 elements of the array. (When making a slice, the capacity can be omitted; see the section on slices for more information.) In contrast, `new([]int)` returns a pointer to a newly allocated, zeroed slice structure, that is, a pointer to a `nil` slice value.

These examples illustrate the difference between `new` and `make`.

```
var p *[]int = new([]int)    // allocates slice structure; *p == nil; rarely useful
var v []int = make([]int, 100) // the slice v now refers to a new array of 100 ints

// Unnecessarily complex:
var p *[]int = new([]int)
*p = make([]int, 100, 100)

// Idiomatic:
v := make([]int, 100)
```

<https://goweexamples.com/mysql-database/>

<https://rezasi.github.io/go-interview-practice/fiber>

- <https://github.com/RezaSi/go-interview-practice/blob/main/packages/fiber/challenge-1-basic-routing/solution-template.go>
- <https://github.com/RezaSi/go-interview-practice/blob/main/packages/fiber/challenge-1-basic-routing/submissions/RezaSi/solution.go>

NOTA: em vez de fazer assim (meu rascunho até aprender a lidar melhor):

```
for rows.Next() {
    var p models.Produto
    err := rows.Scan(&p.ID, &p.Nome)
    if err != nil {
        log.Fatal(err)
    }
}
```

é assim que fazemos no fiber para evitar quebrar o programa inteiro porque uma query falhou ou algo do tipo:

```
found
app.Get("/tasks/:id", func(c fiber.Ctx) error {
    idStr := c.Params("id")
    id, err := strconv.Atoi(idStr)
    if err != nil {
        return c.Status(400).JSON(fiber.Map{"error": "Invalid task ID"})
    }
})
```

NOTA:

```
// GetAll returns all tasks
func (ts *TaskStore) GetAll() []*Task {
    ts.mu.RLock()
    defer ts.mu.RUnlock()

    ✨ tasks := make([]*Task, 0, len(ts.tasks))
    for _, task := range ts.tasks {
        tasks = append(tasks, task)
    }
    return tasks
}
```

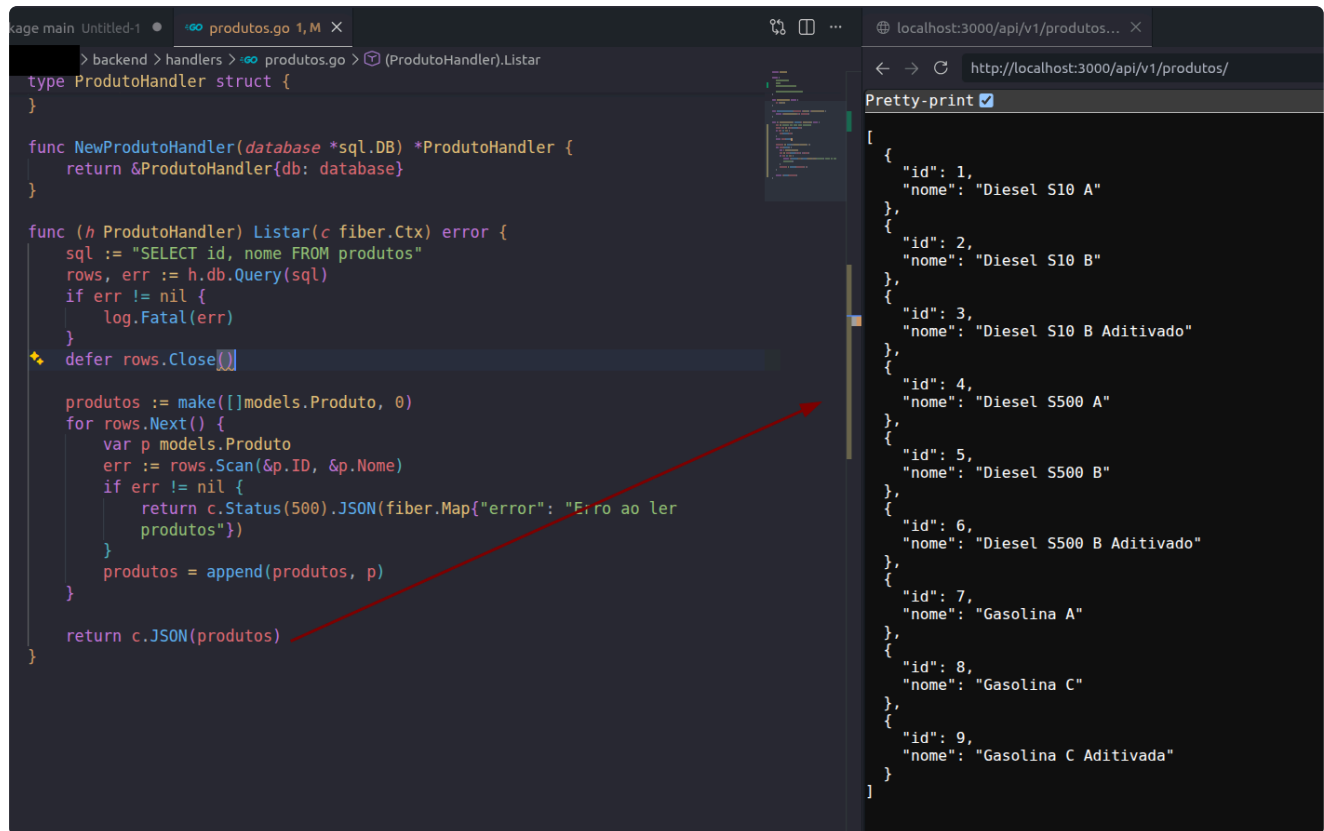
vi esse padrão nesse exemplo e em alguns videos, e fiquei confuso quando a diferença:

```
var produtos []models.Produto
for rows.Next() {
    // ....
}

produtos := make([]models.Produto, 0)
for rows.Next() {
    // ....
}
```

além da diferença de pré-alocar o slice na memória, *"they signal different intentions to other developers."*

NICE:



The screenshot shows a Go IDE with a file named `produtos.go`. The code defines a `ProdutoHandler` struct and a `Listar` function. The `Listar` function queries a database for products and returns a JSON array. A red arrow points from the `return c.JSON(produtos)` line in the code to the JSON output in the browser. The browser shows a list of 9 products with their IDs and names.

```
type ProdutoHandler struct {
}

func NewProdutoHandler(database *sql.DB) *ProdutoHandler {
    return &ProdutoHandler{db: database}
}

func (h ProdutoHandler) Listar(c fiber.Ctx) error {
    sql := "SELECT id, nome FROM produtos"
    rows, err := h.db.Query(sql)
    if err != nil {
        log.Fatal(err)
    }
    defer rows.Close()

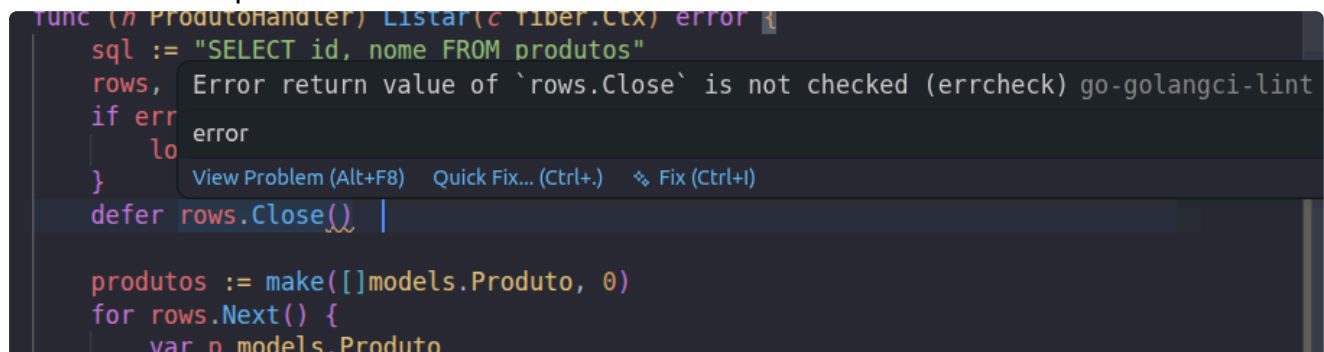
    produtos := make([]models.Produto, 0)
    for rows.Next() {
        var p models.Produto
        err := rows.Scan(&p.ID, &p.Nome)
        if err != nil {
            return c.Status(500).JSON(fiber.Map{"error": "Erro ao ler produtos"})
        }
        produtos = append(produtos, p)
    }

    return c.JSON(produtos)
}
```

localhost:3000/api/v1/produtos/

```
[
  {
    "id": 1,
    "nome": "Diesel S10 A"
  },
  {
    "id": 2,
    "nome": "Diesel S10 B"
  },
  {
    "id": 3,
    "nome": "Diesel S10 B Aditivado"
  },
  {
    "id": 4,
    "nome": "Diesel S500 A"
  },
  {
    "id": 5,
    "nome": "Diesel S500 B"
  },
  {
    "id": 6,
    "nome": "Diesel S500 B Aditivado"
  },
  {
    "id": 7,
    "nome": "Gasolina A"
  },
  {
    "id": 8,
    "nome": "Gasolina C"
  },
  {
    "id": 9,
    "nome": "Gasolina C Aditivada"
  }
]
```

mas tem isso aqui



The screenshot shows a Go IDE with a linting error. The error message is "Error return value of `rows.Close` is not checked (errcheck) go-golangci-lint". The error is located on the line `defer rows.Close()`. The IDE suggests a quick fix to add an error check.

```
func (h ProdutoHandler) Listar(c fiber.Ctx) error {
    sql := "SELECT id, nome FROM produtos"
    rows, err := h.db.Query(sql)
    if err != nil {
        log.Fatal(err)
    }
    defer rows.Close()

    produtos := make([]models.Produto, 0)
    for rows.Next() {
        var p models.Produto
```

na vdd vou colar o return status 500 quando dá erro NA QUERY, nao na iteração do Scan()

```
func (h ProdutoHandler) Listar(c fiber.Ctx) error {
    sql := "SELECT id, nome FROM produtos"
    rows, err := h.db.Query(sql)
    if err != nil {
        return c.Status(500).JSON(fiber.Map{"error": "Erro ao consultar produtos"})
    }
    defer rows.Close()

    produtos := make([]models.Produto, 0)
    for rows.Next() {
        var p models.Produto
        err := rows.Scan(&p.ID, &p.Nome)
        if err != nil {
            continue
        }
        produtos = append(produtos, p)
    }

    return c.JSON(produtos)
}
```

agora fica acusando erro no rows.Close()

Error return value of rows.Close is not checked (errcheck)go-golangci-lint error

é porque o rows.Close() também retorna erro, mas quando ele retorna erro a conexão já deu ruim... então meio que não faz diferença

PORÉM a ideia do Go é sempre tratar erros explicitamente né... ou "se for ignorar, ignore explicitamente" pra ficar claro que foi uma decisão e não um oversight

aliás, da própria documentação:

Handling errors

Be sure to check for an error from sql.Rows after looping over query results. If the query failed, this is how your code finds out.

column values into variables represented by pointers.

```
func albumsByArtist(artist string) ([]Album, error) {
    rows, err := db.Query("SELECT * FROM album WHERE artist = ?", artist)
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    // An album slice to hold data from returned rows.
    var albums []Album

    // Loop through rows, using Scan to assign column data to struct fields.
    for rows.Next() {
        var alb Album
        if err := rows.Scan(&alb.ID, &alb.Title, &alb.Artist,
            &alb.Price, &alb.Quantity); err != nil {
            return albums, err
        }
        albums = append(albums, alb)
    }
    if err = rows.Err(); err != nil {
        return albums, err
    }
    return albums, nil
}
```

Note the deferred call to `rows.Close`. This releases any resources held by the rows no mat

```

rows, err := db.Query("SELECT * from album; SELECT * from song;")
if err != nil {
    log.Fatal(err)
}
defer rows.Close()

// Loop through the first result set.
for rows.Next() {
    // Handle result set.
}

// Advance to next result set.
rows.NextResultSet()

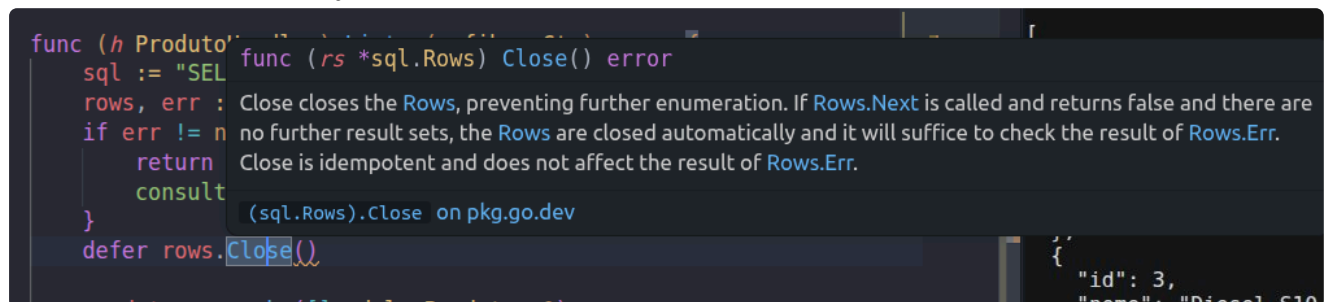
// Loop through the second result set.
for rows.Next() {
    // Handle second set.
}

// Check for any error in either result set.
if err := rows.Err(); err != nil {
    log.Fatal(err)
}

```

legal.... o linter reclama do rows.Close() mas na documentação não tratam o erro do Close()??

ah... *"Rows.Next is called and returns false and there are no further result sets, the Rows are closed automatically and it will suffice to check the result of Rows.Err."*



The screenshot shows a Go code snippet in an IDE. The code is:

```

func (h Produto) ...
sql := "SEL
rows, err :
if err != n
    return
consult
}
defer rows.Close()

```

A tooltip for the `Close()` method is displayed, showing the following text:

```

func (rs *sql.Rows) Close() error
Close closes the Rows, preventing further enumeration. If Rows.Next is called and returns false and there are no further result sets, the Rows are closed automatically and it will suffice to check the result of Rows.Err. Close is idempotent and does not affect the result of Rows.Err.
(sql.Rows).Close on pkg.go.dev

```

bom, tratei o rows.Err no final, mas o linter ainda reclama. Porém parece que é normal usar só assim mesmo

https://www.reddit.com/r/golang/comments/yrgths/how_to_deal_error_caused_by_close/

<https://www.joeshaw.org/dont-defer-close-on-writable-files/>

Is it "normal" to ignore it? Yes.

Is it "safe" to ignore it? In the case of `rows.Close()`, yes.

Should you ignore it in a linter-heavy project? No, just use the blank identifier or a `nolint` comment to keep the "red squiggly lines" away.

Does it feel a bit silly to wrap a simple line in an anonymous function just to please a tool?

Welcome to Go! We trade a little bit of boilerplate for a lot of reliability.

aparentemente podemos explicitar que estamos ignorando, como

```
defer func() { _ = rows.Close() }()
```

ou simplesmente ignorar no caso de LEITURA. É mais importante se preocupar com isso quando envolve ESCRITA de alguma coisa

[^e66932](#) eu falei isso, mas é um grande erro

porque se dá algum problema no meio da query, a chamada pode retornar resultados parciais, e o usuário da api não vai ter como saber. NESSE CASO o mais importante é a integridade dos dados

melhor assim:

```
produtos := make([]models.Produto, 0)
for rows.Next() {
    var p models.Produto
    err := rows.Scan(&p.ID, &p.Nome)
    if err != nil {
        return c.Status(500).JSON(fiber.Map{"error": "Erro ao ler produto do banco de dados"})
    }
    produtos = append(produtos, p)
}
```

as mensagens de erro estavam todas muito similares, e apesar de não fazer tanta diferença para um usuário onde EXATAMENTE foi o erro, para o debug é essencial

daí entrei numa investigação sobre como escrever mensagens de erro em apis

a "regra" é ser vago com o cliente (até por questão de segurança, pra não expor demais para um possível hacker) e específico com o desenvolvedor

O que me fez me tocar que tinha passao a mandar só o JSON e não estava mandando nada para o log interno

vou seguir a ideia de:

```
if err != nil {
    // log bem específico para debug
```

```

log.Printf("[ERROR] Query failed: %v", err)

// retorno mais genérico para o cliente
return c.Status(500).JSON(fiber.Map{"error": "Não foi possível
buscar os produtos no momento."})
}

```

acredito que finalizei a listagem dos produtos.

quando voltar, implementar busca dos motoristas:

```

package handlers

import (
    "database/sql"
    "github.com/gofiber/fiber/v3"
)

type MotoristaHandler struct {
    db *sql.DB
}

func NewMotoristaHandler(database *sql.DB) *MotoristaHandler {
    return &MotoristaHandler{db: database}
}

func (h *MotoristaHandler) Buscar(c fiber.Ctx) error {
    // vai exigir algum parâmetro 'cpf' ou 'placa'
    // podemos usar um query param "?", tipo: /motoristas/busca?cpf=XXX OU busca?
    // placa=XXX
    // fazer uma checagem primeiro pra validar se a placa é realmente uma placa, ou se o
    // cpf é um cpf válido... ou isso fica só no front? (NAO)
    // se bem que nao da pra confiar só no front, porque não impediria de fazer um
    // request direto pra api
    return c.SendString("Busca motorista por CPF ou placa para pré-preenchimento do
    formulário ")
}

package routes

import (
    "database/sql"
    "github.com/gofiber/fiber/v3"
)

func SetupRoutes(app *fiber.App, database *sql.DB) {
    produto := handlers.NewProdutoHandler(database)
    motorista := handlers.NewMotoristaHandler(database)
    fila := handlers.NewFilaHandler(database)

    api := app.Group("/api/v1")

    // Produtos
    api.Get("/produtos", produto.Listar)

    // Motoristas
    api.Get("/motoristas/busca", motorista.Buscar)

    // Fila
    api.Get("/fila/historico", fila.Historico)
    api.Get("/fila/:id", fila.BuscarEntrada)
    api.Get("/fila", fila.ListarEntradas)
    api.Patch("/fila/:id/status", fila.AtualizarStatus)
    api.Post("/fila", fila.CriarEntrada)
}

```

(09/05)

<https://medium.com/@isharapeiris2001/query-parameters-vs-api-route-parameters-using-golang-931ad87151dd>

<https://medium.com/@isharapeiris2001/query-parameters-vs-api-route-parameters-using-golang-931ad87151dd>

estamos chegando na deadline, preciso dar uma acelerada, então vou parar de ser tão preciosista nesse documento (até porque já consigo me virar melhor agora com o que já sei de Go e as fontes que tenho para consultar)

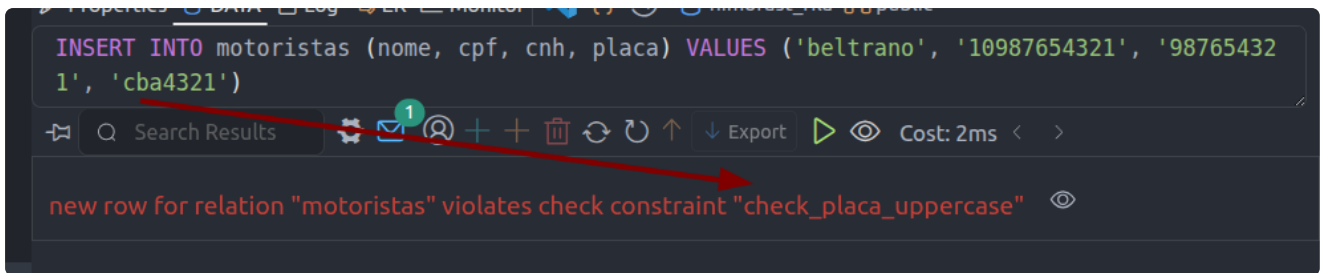
implementei a função de busca de motorista

OBS PRA LEMBRAR: regex declarado fora das funções para não ter que processar toda vez que houver um request

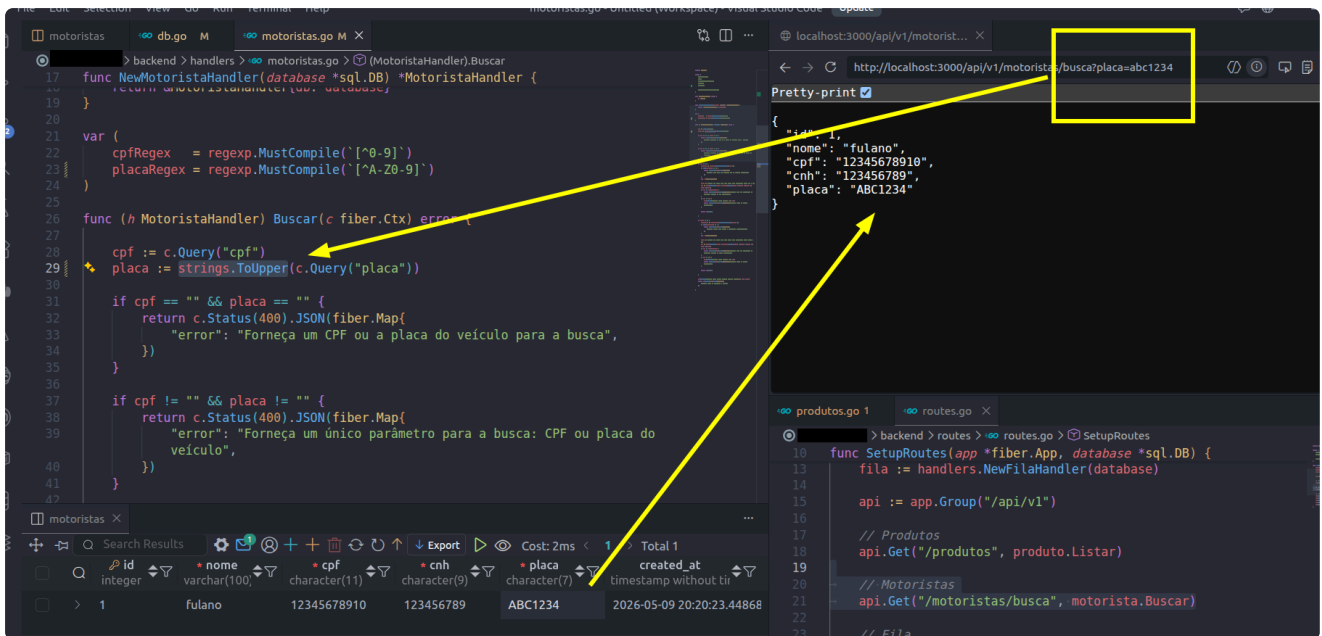
NOTA:

- quebrar a função "Buscar"? (separar a busca de cpf e placa) -> a função faz tudo dentro dela
- garantir que as letras da placa são sempre MAIUSCULAS
 - banco: não aceitar minúscula? Trigger para converter direto?
 - api: passar o query param para uppercase

no momento, optei por usar um CONSTRAINT para garantir que não aceita lowercase e depois na api usar os inserts com UPPER('value')



beleza:



acho que finalizei a parte dos motoristas

para finalizar o backend, pelo menos por agora, faltam as funções do handler da fila. Vamo que vamo

```
db.go motoristas.go fila.go M
> backend > handlers > fila.go > (FilaHandler).CriarEntrada

9 type FilaHandler struct {
11 }
12
13 func NewFilaHandler(database *sql.DB) *FilaHandler {
14     return &FilaHandler{db: database}
15 }
16
17 func (h FilaHandler) CriarEntrada(c fiber.Ctx) error {
18     // do pdf - "Entrada na Fila – referência ao motorista,
19     // produto solicitado, status atual, horário de chegada, início e
20     // finalização do carregamento"
21
22     // é a "entrada que manda"
23     // exige todos os dados do motorista e o produto que ele
24     // veio buscar
25     // a partir daqui é que será feita a criação de um motorista
26
27     // esses dados virão de um form no front que será enviado via
28     // POST
29
30     return c.SendString("Registra a chegada de um motorista na fila")
31 }
```

MINA DE OURO AQUI: <https://dev.to/koddr/build-a-restful-api-on-go-fiber-postgresql-jwt-and-swagger-docs-in-isolated-docker-containers-475j>

The image shows two screenshots of code from a Dev.to article. The left screenshot shows Go code for creating a book, with a purple arrow pointing to the `book := models.Book{}` line. The right screenshot shows the `models` package definition, with a purple box highlighting the `Book` struct and its fields.

```
// Create new Book struct
book := models.Book{}

// Check, if received JSON data is valid
if err := c.BodyParser(&book); err != nil {
    // Return status 400 and error message
    return c.Status(fiber.StatusBadRequest).JSON(fiber.Map{
        "error": true,
        "msg": err.Error(),
    })
}

// Create database connection.
db, err := database.OpenDBConnection()
if err != nil {
    // Return status 500 and database connection error.
    return c.Status(fiber.StatusInternalServerError).JSON(fiber.Map{
        "error": true,
        "msg": err.Error(),
    })
}

// Create a new validator for a Book model.
validate := utils.NewValidator()

// Set initialized default data for book:
book.ID = uuid.New()
book.CreatedAt = time.Now()
book.BookStatus = 1 // 0 == draft, 1 == active

// Validate book fields.
if err := validate.Struct(&book); err != nil {
    // Return, if some fields are not valid.
    return c.Status(fiber.StatusBadRequest).JSON(fiber.Map{
        "error": true,
        "msg": utils.ValidatorErrors(err),
    })
}

// Create book.
if err := db.CreateBook(&book); err != nil {
    // Return status 500 and error message.
    return c.Status(fiber.StatusInternalServerError).JSON(fiber.Map{
        "error": true,
        "msg": err.Error(),
    })
}
```

```
// app/models/book_model.go

package models

import (
    "database/sql/driver"
    "encoding/json"
    "errors"
    "time"

    "github.com/google/uuid"
)

// Book struct to describe book object.
type Book struct {
    ID          uuid.UUID `db:"id" json:"id" validate:"required"`
    CreatedAt   time.Time `db:"created_at" json:"created_at"`
    UpdatedAt   time.Time `db:"updated_at" json:"updated_at"`
    UserID      uuid.UUID `db:"user_id" json:"user_id" validate:"required"`
    Title       string    `db:"title" json:"title" validate:"required"`
    Author      string    `db:"author" json:"author" validate:"required"`
    BookStatus  int       `db:"book_status" json:"book_status"`
    BookAttrs   BookAttrs `db:"book_attrs" json:"book_attrs"`
}

// BookAttrs struct to describe book attributes.
type BookAttrs struct {
    Picture string `json:"picture"`
    Description string `json:"description"`
    Rating int `json:"rating" validate:"min=1,max=5"`
}

// ...
```

👉 I recommend to use the [google/uuid](#) package to create unique IDs, because this is a more versatile way to protect your application against common number brute force attacks. Especially if your REST API will have public methods without authorization and request limit.

mas o meu model EntradaFila é mais complexo que o que eu realmente preciso para a função de criar entrada

<https://github.com/bxcodec/go-clean-arch>

<https://medium.easyread.co/golang-clean-architecture-efd6d7c43047>

Título

- **Camada de Transporte (Transport/API Layer):** É onde residem seus structs de request. Eles definem o que a API aceita. Eles costumam ter tags de serialização (ex: `json:"nome"`) e tags de validação (ex: `validate:"required"` usando bibliotecas como `go-playground/validator`).
- **Camada de Domínio (Domain/Service Layer):** É onde residem suas entidades. Elas representam a lógica de negócio pura e a estrutura persistida no banco de dados. Elas não devem "saber" sobre o protocolo HTTP ou formatos de JSON.

ou seja, normal criar aqui uma nova struct específica para o TRANSPORTE

sobre o `validate: required` -> estava esquecendo desse conceito. Como `fiber` é inspirado no `express`, imagino que siga a mesma lógica como fazia com o `express-validator`

aparentemente é usada a biblioteca `"go-playground/validator"` mencionada no callout acima

... como está chegando na deadline, vou deixar isso como to-do

com essa struct criada, preciso agora instanciar uma entidade na função de `CriarEntrada`

```
type criarEntrada struct {
    Nome      string `json:"nome"`
    CPF       string `json:"cpf"`
    CNH       string `json:"cnh"`
    Placa     string `json:"placa"`
    ProdutoID int    `json:"produto_id"`
}
```

OBS: não é mais `c.BodyParser`

<https://docs.gofiber.io/blog/fiber-v3-binding-in-practice/#what-v2-parsing-looked-like>

...oh no, só aprendi agora que

```
// isso aqui
err := c.Bind().Body(&ce)
if err != nil {
    return c.Status(400).JSON(fiber.Map{"error": "Erro ao processar os dados fornecidos"})
} // err ainda existe seguindo esse bloco

// é diferente disso
err := c.Bind().Body(&ce)
```

```

    if err := c.Bind().Body(&ce); err != nil {
        return c.Status(400).JSON(fiber.Map{"error": "Erro ao processar os dados fornecidos"})
    } // err já não existe mais -> fica limitado a esse bloco

```

(10/05)

aqui tem um ponto de observação: o fato de "placa" ser um atributo do MOTORISTA gera algumas limitações

porque as entradas registram o id do motorista, mas se queremos ver algum tipo de historico, e o motorista mudou de veiculo, nao conseguiriamos saber qual realmente foi o veiculo que foi carregado com o produto

para efeitos de simplicidade, estou considerando que a placa é imutavel assim como os documentos do motorista. Se fossemos permitir essa troca, teriamos que analisar algumas possibilidades

- nova row para o mesmo motorista (mesmo cpf e cnh, placa diferente) -> nao parece um bom modelo do mundo real. Devemos ter 1 id para cada motorista
- salvar placa numa coluna na entrada por motivos de historico
- **ou, talvez o que modele melhor o mundo real, ter uma tabela de veículos, e na tabela trocar a coluna placa por um link com veículos, permitindo que um motorista vincule diferentes veículos, e salvar o id do veiculo na entrada, assim como o id do motorista**

(coloquei nos to-dos)

não entendi porque não está indo... minha lógica é que o parâmetro tentaria ser convertido para int -> se é vazio ou não numerico, ficaria 0 e cairia no erro, mas quando tento 1,2,3 não funciona também...

The screenshot shows a code editor with the following Go code in `handlers.go`:

```

func (h FilaHandler) BuscarEntrada(c fiber.Ctx) error {
    entradaID := fiber.Query[int](c, "id")
    if entradaID == 0 {
        return c.Status(400).JSON(fiber.Map{"error": "0 ID da entrada é obrigatório e deve ser um número inteiro"})
    }

    var e models.EntradaFila

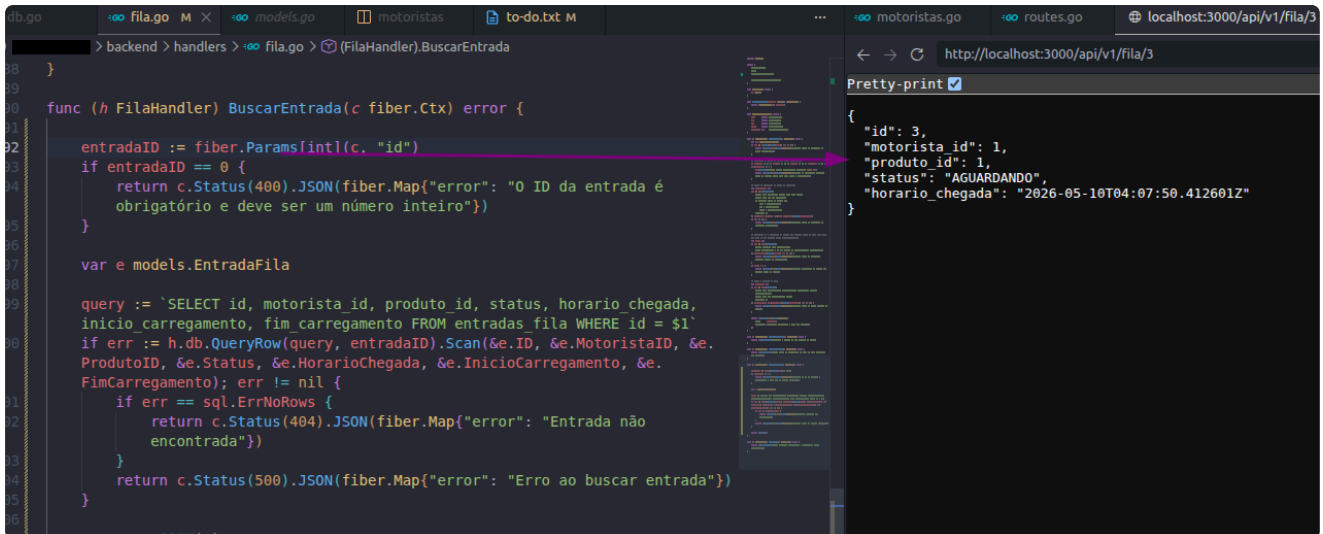
    query := `SELECT id, motorista_id, produto_id, status, horario chegada,
    inicio carregamento, fim carregamento FROM entradas_fila WHERE id = $1`
    if err := h.db.QueryRow(query, entradaID).Scan(&e.ID, &e.MotoristaID, &e.
    ProdutoID, &e.Status, &e.HorarioChegada, &e.InicioCarregamento, &e.
    FimCarregamento); err != nil {
        if err == sql.ErrNoRows {
            return c.Status(404).JSON(fiber.Map{"error": "Entrada não encontrada"})
        }
        return c.Status(500).JSON(fiber.Map{"error": "Erro ao buscar entrada"})
    }

    return c.JSON(e)
}

```

The browser at `http://localhost:3000/api/v1/fila/3` shows a 400 error response: `{\"error\": \"0 ID da entrada é obrigatório e deve ser um número inteiro\"}`. A red arrow points from the error message to the `id` parameter in the code, and a red question mark is next to the error message.

AH: é porque estou tentando ler um QUERY param, mas é um ROUTE param nesse caso (diferente das outras funções que implementei até aqui) -> é só trocar Query por Params



The screenshot shows the `BuscarEntrada` function in `fila.go`. A red arrow points from the `fiber.Params["id"]` line to the browser's address bar, which shows `http://localhost:3000/api/v1/fila/3`. The browser's 'Pretty-print' tab shows the JSON response:

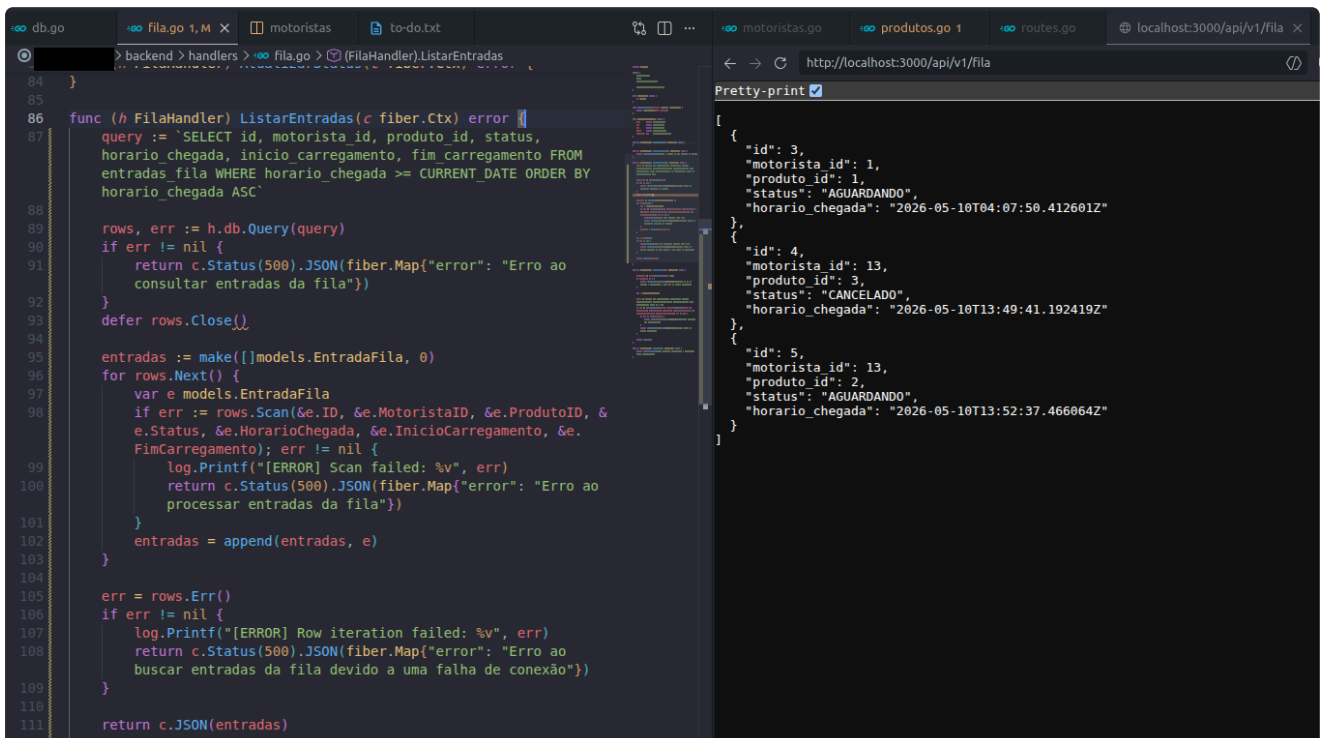
```
{
  "id": 3,
  "motorista_id": 1,
  "produto_id": 1,
  "status": "AGUARDANDO",
  "horario_chegada": "2026-05-10T04:07:50.412601Z"
}
```

NOTA: sobre ListarEntradas, estava matutando como lidaria com a questão da data (listar somente entradas do dia atual)

Descobri que tem uma forma muito simples direto pelo SQL pra não ter problemas com fuso e nada disso

```
WHERE horario_chegada >= CURRENT_DATE
```

(CURRENT_DATE pega a data de hoje "sem horario" (à meia noite), então realmente só vai pegar entradas criadas hoje)



The screenshot shows the `ListarEntradas` function in `fila.go`. The browser's address bar shows `http://localhost:3000/api/v1/fila`. The 'Pretty-print' tab shows the JSON response:

```
[
  {
    "id": 3,
    "motorista_id": 1,
    "produto_id": 1,
    "status": "AGUARDANDO",
    "horario_chegada": "2026-05-10T04:07:50.412601Z"
  },
  {
    "id": 4,
    "motorista_id": 13,
    "produto_id": 3,
    "status": "CANCELADO",
    "horario_chegada": "2026-05-10T13:49:41.192419Z"
  },
  {
    "id": 5,
    "motorista_id": 13,
    "produto_id": 2,
    "status": "AGUARDANDO",
    "horario_chegada": "2026-05-10T13:52:37.466864Z"
  }
]
```

BOA

historico: só copiei a função de listar e mudei a query para pegar entradas com `horario_chega < hoje` e ordenar de forma decrescente (mais recentes primeiro)

Dupliquei a row de id 5 duas vezes e alterei o dia da data manualmente pra testar:

The screenshot shows the Visual Studio Code interface with a database query editor on the left and a JSON viewer on the right. The query is `SELECT * FROM entradas_fila LIMIT 100`. The results table has columns: id, motorista_id, produto_id, status, horario_chegada, and inicio_carregamento. The JSON viewer shows the response from `http://localhost:3000/api/v1/fila/historico`, which is an array of objects containing the same data as the table.

id	motorista_id	produto_id	status	horario_chegada	inicio_carregamento
3	1	1	AGUARDANDO	2026-05-10 04:07:50.41260	(NULL)
4	13	3	CANCELADO	2026-05-10 13:49:41.19241	(NULL)
5	13	2	AGUARDANDO	2026-05-09 13:52:37.46606	(NULL)
6	13	2	AGUARDANDO	2026-05-09 13:52:37.46606	(NULL)
7	13	2	AGUARDANDO	2026-05-08 13:52:37.46606	(NULL)

AH, DETALHE: é pra mostrar só CANCELADAS E FINALIZADAS. Vou corrigir

The screenshot shows the Visual Studio Code interface with a database query editor on the left and a JSON viewer on the right. The query is `SELECT * FROM entradas_fila LIMIT 100`. The results table has columns: id, motorista_id, produto_id, status, horario_chegada, and inicio_carregamento. The JSON viewer shows the response from `http://localhost:3000/api/v1/fila/historico`, which is an array of objects containing the same data as the table. The status column is highlighted in the table, showing 'AGUARDANDO', 'CANCELADO', and 'FINALIZADO'.

id	motorista_id	produto_id	status	horario_chegada	inicio_carregamento
3	1	1	AGUARDANDO	2026-05-10 04:07:50.41260	(NULL)
4	13	3	CANCELADO	2026-05-10 13:49:41.19241	(NULL)
5	13	2	AGUARDANDO	2026-05-09 13:52:37.46606	(NULL)
6	13	2	FINALIZADO	2026-05-09 13:52:37.46606	(NULL)
7	13	2	CANCELADO	2026-05-08 13:52:37.46606	(NULL)

Agora estou trabalhando na `AtualizarStatus()`

como posso fazer a validação da troca de status? (só AGUARDANDO e CARREGANDO podem mudar)

como já estou usando o type Status dentro da função, posso implementar algum método próprio para o type Status

The screenshot shows the Visual Studio Code interface with a Go file named `models.go`. The code defines a `Status` type and constants for different status values.

```
package models

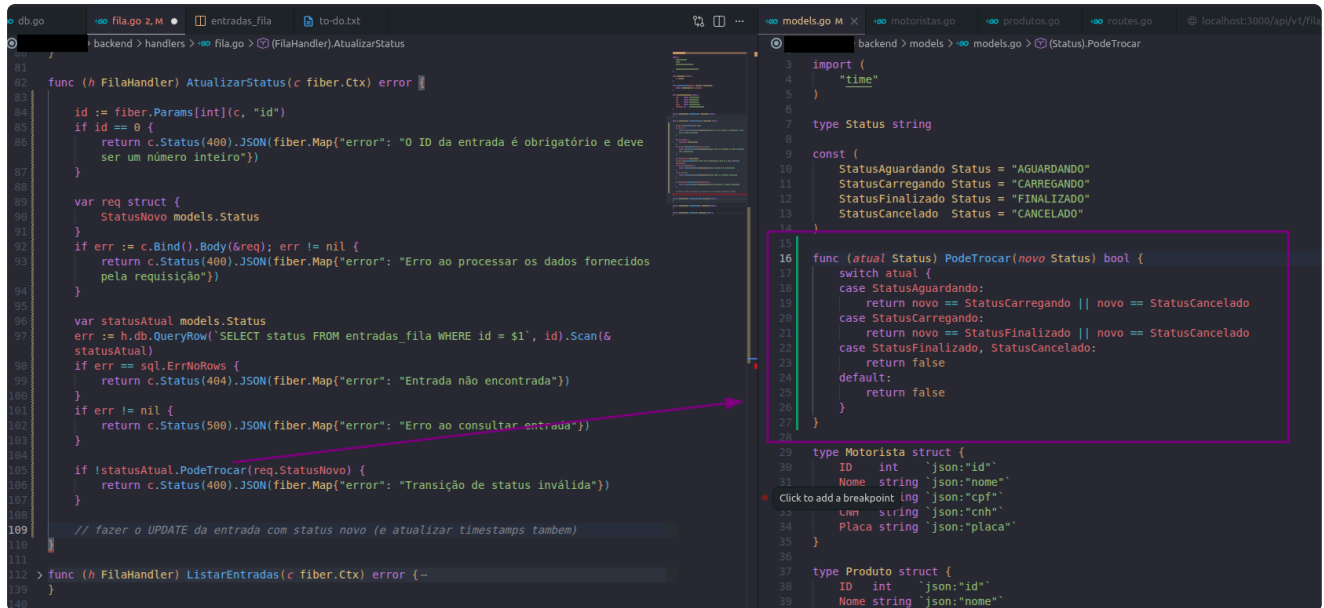
import (
    "time"
)

type Status string

const (
    StatusAguardando Status = "AGUARDANDO"
    StatusCarregando Status = "CARREGANDO"
    StatusFinalizado Status = "FINALIZADO"
    StatusCancelado Status = "CANCELADO"
)

type Motorista struct {
```

um switch já resolve



```
81 func (h FilaHandler) AtualizarStatus(c fiber.Ctx) error {
82
83     id := fiber.Params[int](c, "id")
84     if id == 0 {
85         return c.Status(400).JSON(fiber.Map{"error": "O ID da entrada é obrigatório e deve ser um número inteiro"})
86     }
87
88     var req struct {
89         StatusNovo models.Status
90     }
91     if err := c.Bind().Body(&req); err != nil {
92         return c.Status(400).JSON(fiber.Map{"error": "Erro ao processar os dados fornecidos pela requisição"})
93     }
94
95     var statusAtual models.Status
96     err := h.db.QueryRow("SELECT status FROM entradas_fila WHERE id = $1", id).Scan(&statusAtual)
97     if err == sql.ErrNoRows {
98         return c.Status(404).JSON(fiber.Map{"error": "Entrada não encontrada"})
99     }
100     if err != nil {
101         return c.Status(500).JSON(fiber.Map{"error": "Erro ao consultar entrada"})
102     }
103
104     if !statusAtual.PodeTrocar(req.StatusNovo) {
105         return c.Status(400).JSON(fiber.Map{"error": "Transição de status inválida"})
106     }
107
108     // fazer o UPDATE da entrada com status novo (e atualizar timestamps também)
109 }
110
111 > func (h FilaHandler) ListarEntradas(c fiber.Ctx) error {
112 }
113 }
```

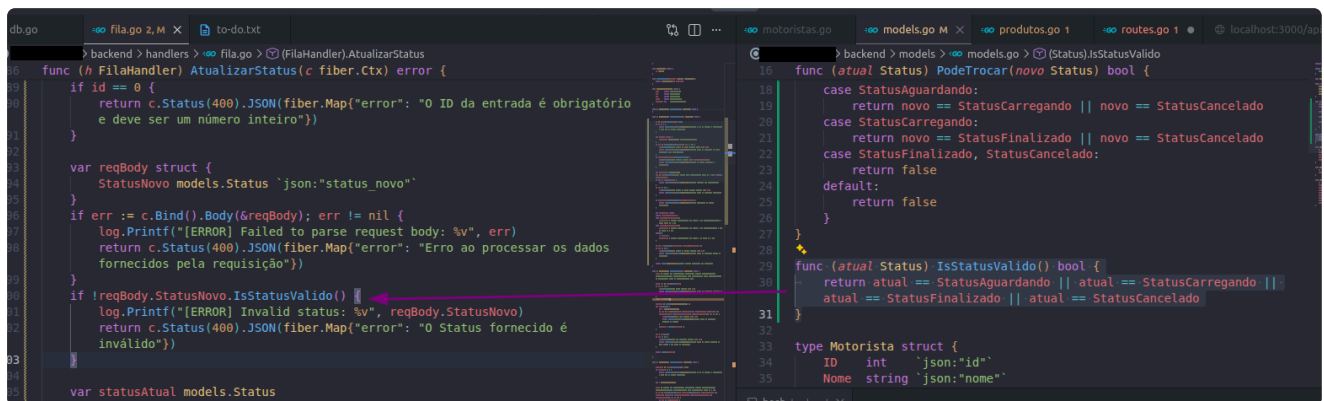
```
3 import (
4     "time"
5 )
6
7 type Status string
8
9 const (
10     StatusAguardando Status = "AGUARDANDO"
11     StatusCarregando Status = "CARREGANDO"
12     StatusFinalizado Status = "FINALIZADO"
13     StatusCancelado Status = "CANCELADO"
14 )
15
16 func (atual Status) PodeTrocar(novo Status) bool {
17     switch atual {
18     case StatusAguardando:
19         return novo == StatusCarregando || novo == StatusCancelado
20     case StatusCarregando:
21         return novo == StatusFinalizado || novo == StatusCancelado
22     case StatusFinalizado, StatusCancelado:
23         return false
24     default:
25         return false
26     }
27 }
```

```
29 type Motorista struct {
30     ID int `json:"id"`
31     Nome string `json:"nome"`
32     Click to add a breakpoint lng `json:"cpf"`
33     Lm string `json:"cnh"`
34     Placa string `json:"placa"`
35 }
36
37 type Produto struct {
38     ID int `json:"id"`
39     Nome string `json:"nome"
40 }
```

falta só a a query de update agora

precisa checar QUAL É a transição -> vai impactar em qual timestamp usar (inicio_carregamento ou fim_carregamento, ou nenhuma se for cancelado)

ESTAVA ESQUECENDO de validar se o status fornecido no PATCH é um status válido



```
6 func (h FilaHandler) AtualizarStatus(c fiber.Ctx) error {
7
8     if id == 0 {
9         return c.Status(400).JSON(fiber.Map{"error": "O ID da entrada é obrigatório e deve ser um número inteiro"})
10     }
11
12     var reqBody struct {
13         StatusNovo models.Status `json:"status_novo"`
14     }
15     if err := c.Bind().Body(&reqBody); err != nil {
16         log.Printf("[ERROR] Failed to parse request body: %v", err)
17         return c.Status(400).JSON(fiber.Map{"error": "Erro ao processar os dados fornecidos pela requisição"})
18     }
19
20     if !reqBody.StatusNovo.IsStatusValido() {
21         log.Printf("[ERROR] Invalid status: %v", reqBody.StatusNovo)
22         return c.Status(400).JSON(fiber.Map{"error": "O Status fornecido é inválido"})
23     }
24
25     var statusAtual models.Status
26     err := h.db.QueryRow("SELECT status FROM entradas_fila WHERE id = $1", id).Scan(&statusAtual)
27     if err == sql.ErrNoRows {
28         return c.Status(404).JSON(fiber.Map{"error": "Entrada não encontrada"})
29     }
30     if err != nil {
31         return c.Status(500).JSON(fiber.Map{"error": "Erro ao consultar entrada"})
32     }
33
34     if !statusAtual.PodeTrocar(reqBody.StatusNovo) {
35         return c.Status(400).JSON(fiber.Map{"error": "Transição de status inválida"})
36     }
37
38     // fazer o UPDATE da entrada com status novo (e atualizar timestamps também)
39 }
```

```
16 func (atual Status) PodeTrocar(novo Status) bool {
17     case StatusAguardando:
18         return novo == StatusCarregando || novo == StatusCancelado
19     case StatusCarregando:
20         return novo == StatusFinalizado || novo == StatusCancelado
21     case StatusFinalizado, StatusCancelado:
22         return false
23     default:
24         return false
25 }
26
27 func (atual Status) IsStatusValido() bool {
28     return atual == StatusAguardando || atual == StatusCarregando ||
29         atual == StatusFinalizado || atual == StatusCancelado
30 }
31
32 type Motorista struct {
33     ID int `json:"id"`
34     Nome string `json:"nome"
```

hm.. também tem a questão de garantir UPPERCASE

posso converter reqBody.StatusNovo direto na função do handler

faz sentido nesse meu cenário que só aqui recebemos status

MAS se fosse algo mais recorrente, pra não ter que ficar convertendo e checando se o status é valido toda vez, poderia usar um parser que combina UPPERCASE + VALIDAÇÃO

aliás, descobri isso - **unmarshalJSON** - (mas para o caso atual acho overkill pq é o único lugar onde tratamos um status vindo do cliente) - <https://danielhilton.medium.com/custom-unmarshallers-in-go-easier-than-you-d-think-a8a9bb76c2e2>

feito -> realizando testes agora (estou sempre usando o id 3 pra facilitar o teste, então também faço alterações manuais nos status, por isso o primeiro print mostra cancelado e o

segundo carregando, que é uma troca de status proibida)

The screenshot shows a Go project in VS Code. The left pane displays the `AtualizarStatus` function in `fila.go`, which checks if a status change is valid and updates the database. The right pane shows the `SetupRoutes` function in `routes.go`, which defines API endpoints for file management. The bottom pane shows a terminal with curl commands for testing the PATCH endpoint.

```
func (h FileHandler) AtualizarStatus(c fiber.Ctx) error {
    reqBody.StatusNovo = models.Status(strings.ToUpper(string(reqBody.StatusNovo)))

    if !reqBody.StatusNovo.IsStatusValido() {
        log.Printf("[ERROR] Invalid status: %v", reqBody.StatusNovo)
        return c.Status(400).JSON(fiber.Map{"error": "O Status fornecido é inválido"})
    }

    var statusAtual models.Status
    err := h.db.QueryRow("SELECT status FROM entradas_fila WHERE id = $1", id).Scan(&statusAtual)
    if err == sql.ErrNoRows {
        return c.Status(404).JSON(fiber.Map{"error": "Entrada não encontrada"})
    }
    if err != nil {
        log.Printf("[ERROR] Failed to query current status: %v", err)
        return c.Status(500).JSON(fiber.Map{"error": "Erro ao consultar entrada"})
    }

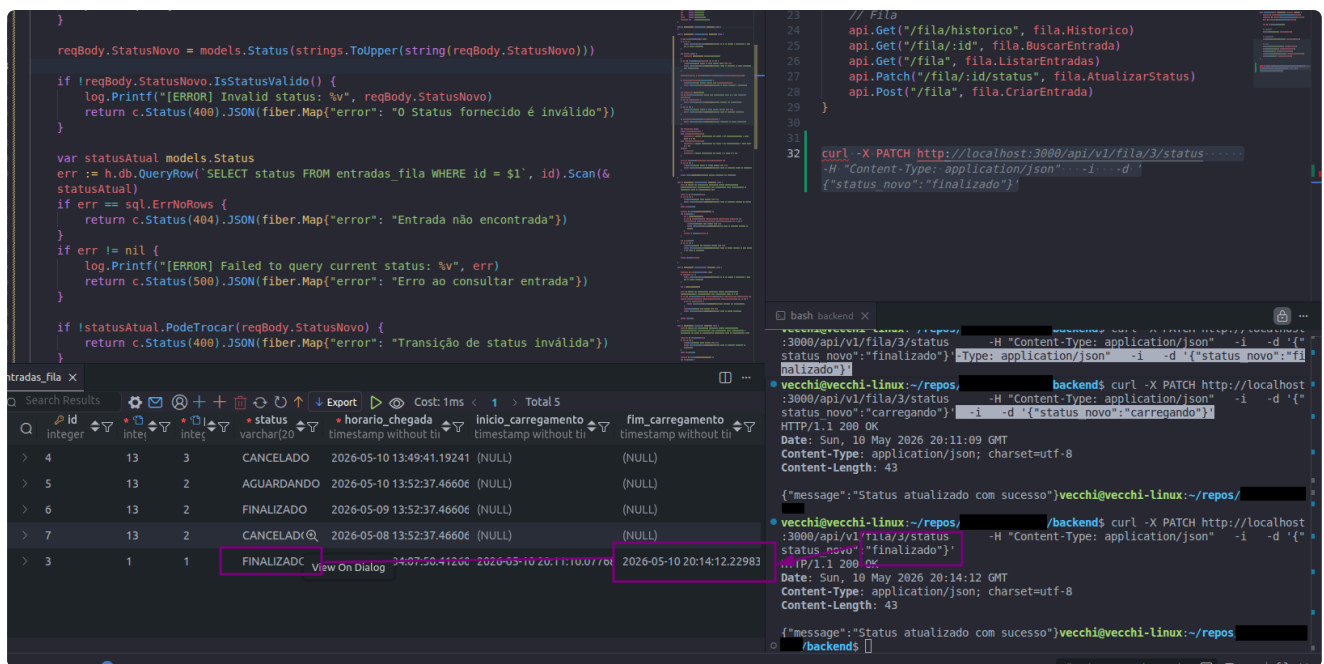
    if !statusAtual.PodeTrocar(reqBody.StatusNovo) {
        return c.Status(400).JSON(fiber.Map{"error": "Transição de status"}
    }
}
```

```
func SetupRoutes(app *fiber.App, database *sql.DB) {
    // Fila
    api.Get("/fila/historico", fila.Historico)
    api.Get("/fila/:id", fila.BuscarEntrada)
    api.Get("/fila", fila.ListarEntradas)
    api.Patch("/fila/:id/status", fila.AtualizarStatus)
    api.Post("/fila", fila.CriarEntrada)
}
```

```
curl -X PATCH http://localhost:3000/api/v1/fila/3/status -H "Content-Type: application/json" -i -d '{"status_novo": "cancelado"}'
```

This screenshot shows the same Go project, but with a different status update logic in `AtualizarStatus`. The `PodeTrocar` method now checks for invalid transitions. The terminal shows a curl command that results in a 400 Bad Request. Below the code, a table displays the state of the `entradas_fila` table.

Id	motorista	produto	status	horario_chegada	inicio_carregamento	fim_carregamento
4	13	3	CANCELADO	2026-05-10 13:49:41.19241	(NULL)	(NULL)
5	13	2	AGUARDANDO	2026-05-10 13:52:37.46606	(NULL)	(NULL)
6	13	2	FINALIZADO	2026-05-09 13:52:37.46606	(NULL)	(NULL)
7	13	2	CANCELADO	2026-05-08 13:52:37.46606	(NULL)	(NULL)
3	1	1	CANCELADO	2026-05-10 04:07:50.41260	2026-05-10 20:11:10.07768	(NULL)



... então acho que finalizei a api por hora

--> passei o backend para a porta 8080 para deixar a 3000 para o next

FINALMENTE FRONTEND

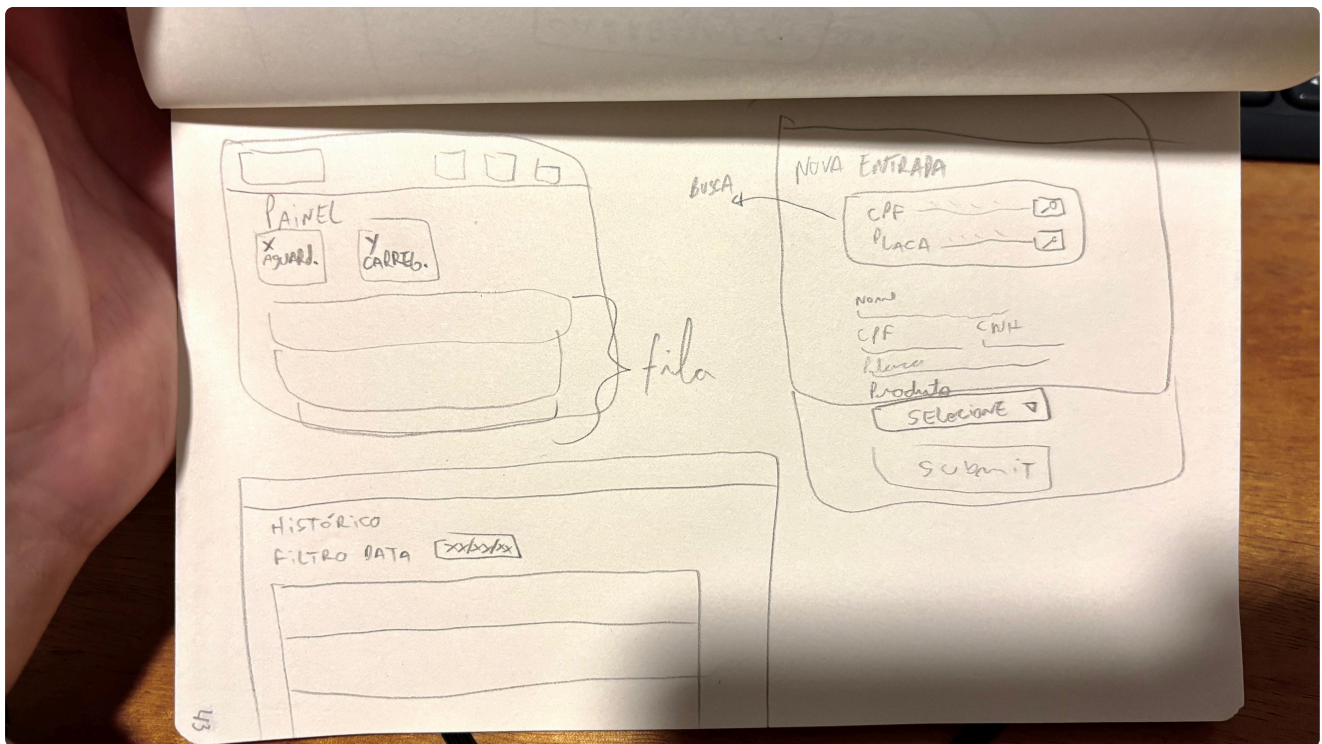
investi quase todo o prazo que eu tinha no backend, mas foi uma decisão consciente:

- um backend bem feito com um front mínimo funciona -> depois é uma questão de ir melhorando
 - mas um front 100% com um backend "furado" não resolve o problema de ninguém
- e apesar de não ser um backend complexo, nunca tinha usado Go
 - mas tinha vontade de aprender, então aproveitei a oportunidade para estudar com a mão na massa
 - mesmo que não seja convocado para a vaga, foi um belo incentivo para dedicar esse tempo e ter essa experiência

Agora vou focar em estruturar um frontend bem simples, porém funcional

inicializei com `npx create-next-app@latest`

como imagino as páginas



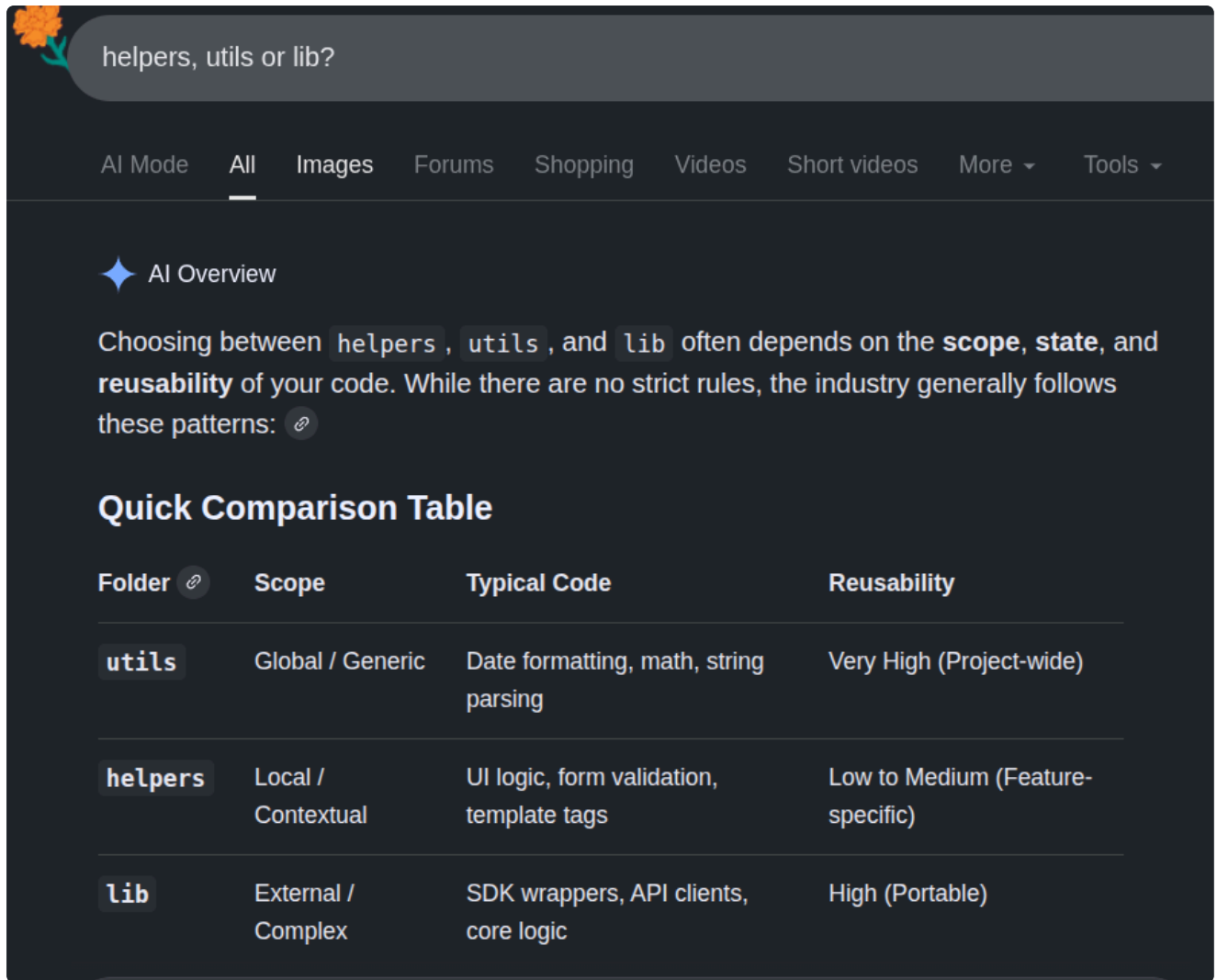
(11/05 5h15)

vendo algo na tela

antes de mais nada, estava tentando fazer um teste simples para checar se a integração front-back estava funcionando

[https://stackoverflow.com/questions/41103360/how-to-use-fetch-in-typescript?
utm_source=chatgpt.com](https://stackoverflow.com/questions/41103360/how-to-use-fetch-in-typescript?utm_source=chatgpt.com)

vou lidar com as requests para a api em um arquivo.... onde coloco?



helpers, utils or lib?

AI Mode All Images Forums Shopping Videos Short videos More Tools

AI Overview

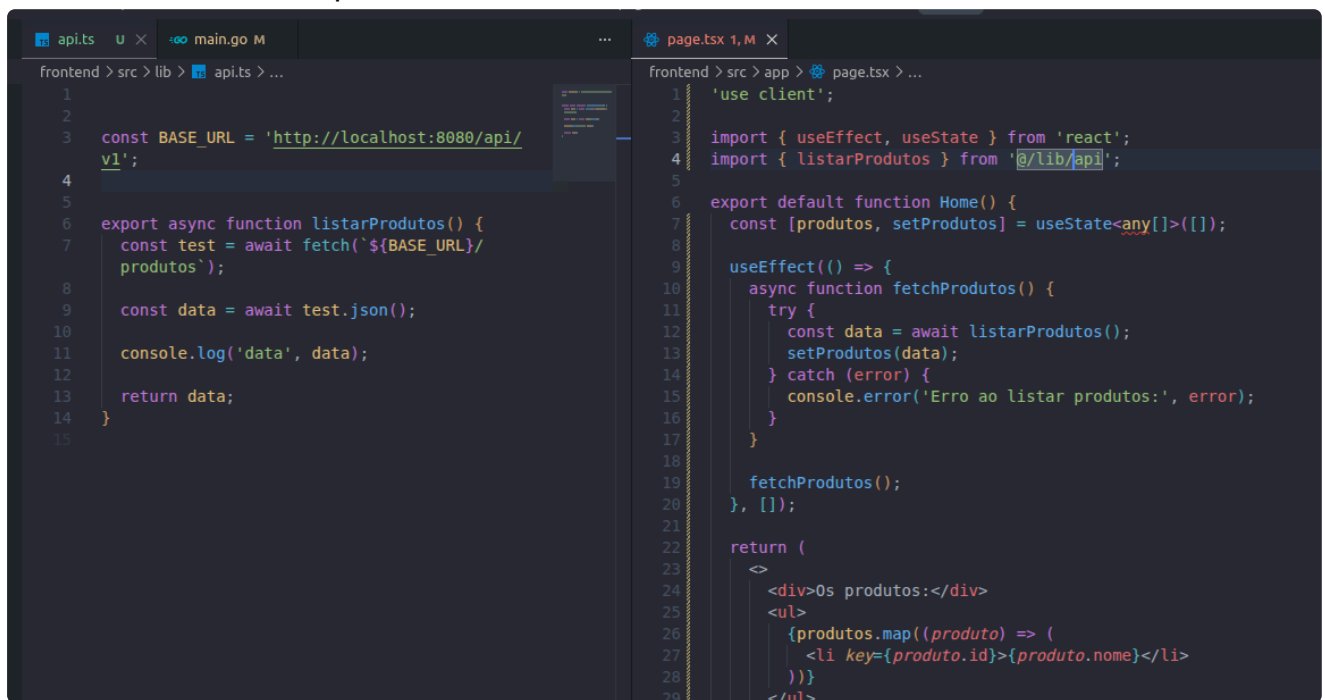
Choosing between **helpers**, **utils**, and **lib** often depends on the **scope**, **state**, and **reusability** of your code. While there are no strict rules, the industry generally follows these patterns: [🔗](#)

Quick Comparison Table

Folder 🔗	Scope	Typical Code	Reusability
utils	Global / Generic	Date formatting, math, string parsing	Very High (Project-wide)
helpers	Local / Contextual	UI logic, form validation, template tags	Low to Medium (Feature-specific)
lib	External / Complex	SDK wrappers, API clients, core logic	High (Portable)

então será lib/api.ts

Queria exibir a lista de produtos na tela:

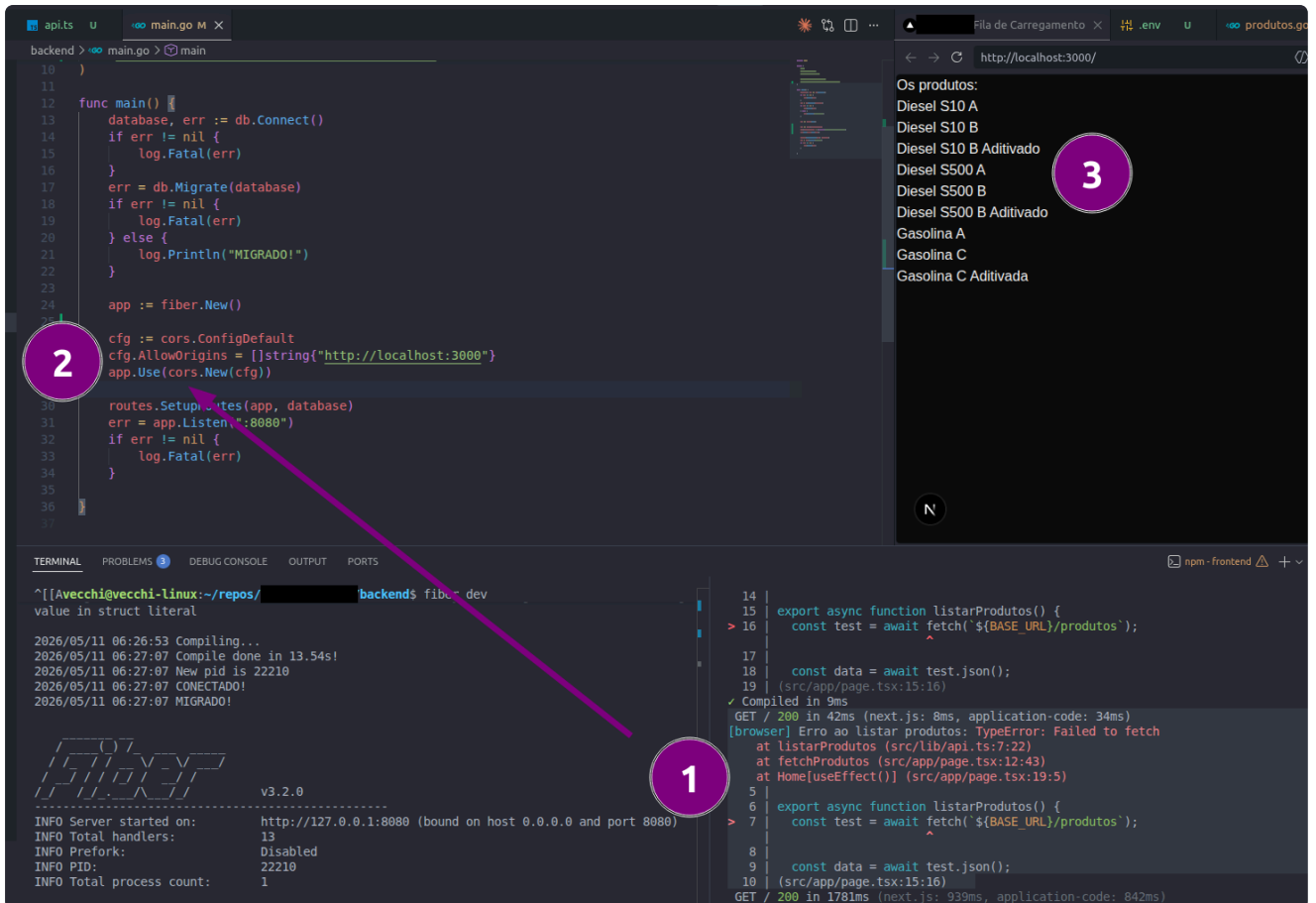


```
api.ts
1
2
3 const BASE_URL = 'http://localhost:8080/api/v1';
4
5
6 export async function listarProdutos() {
7   const test = await fetch(`${BASE_URL}/produtos`);
8
9   const data = await test.json();
10
11   console.log('data', data);
12
13   return data;
14 }
15

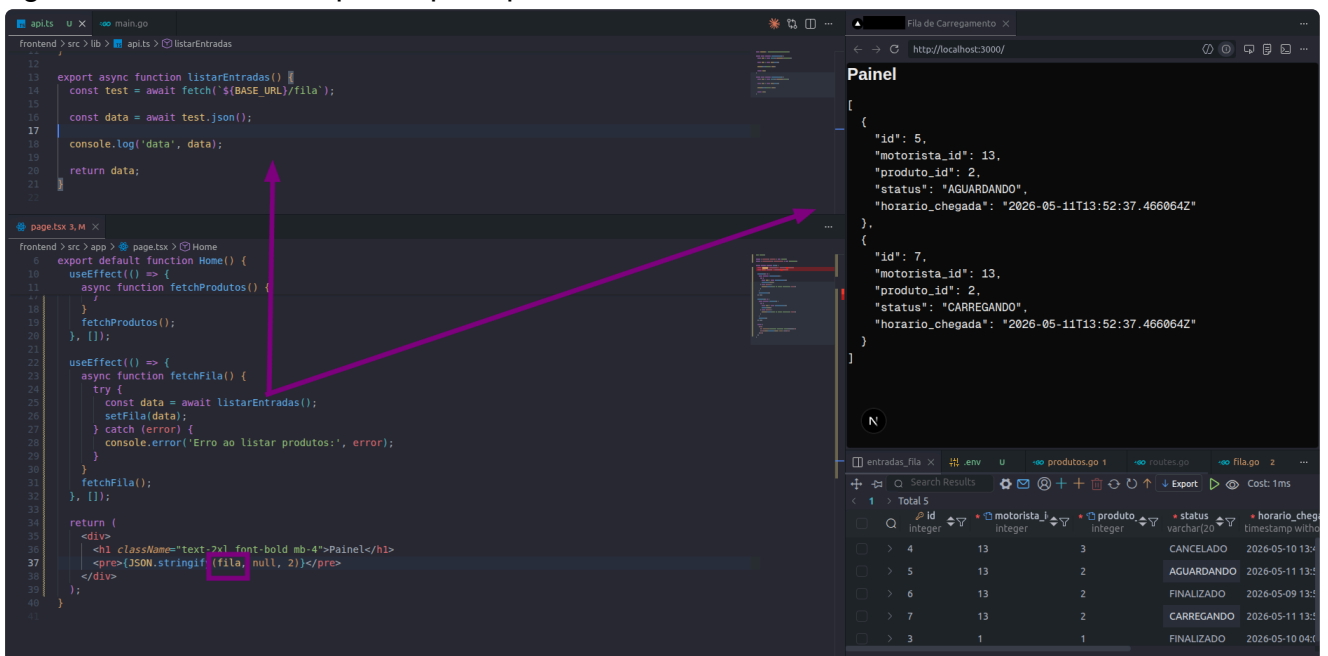
page.tsx
1 'use client';
2
3 import { useEffect, useState } from 'react';
4 import { listarProdutos } from '@lib/api';
5
6 export default function Home() {
7   const [produtos, setProdutos] = useState<any[]>([]);
8
9   useEffect(() => {
10     async function fetchProdutos() {
11       try {
12         const data = await listarProdutos();
13         setProdutos(data);
14       } catch (error) {
15         console.error('Erro ao listar produtos:', error);
16       }
17     }
18
19     fetchProdutos();
20   }, []);
21
22   return (
23     <div>Os produtos:</div>
24     <ul>
25       {produtos.map((produto) => (
26         <li key={produto.id}>{produto.nome}</li>
27       ))}
28     </ul>
29   );
30 }
```

mas estava tendo problema com "Failed to fetch"

precisava habilitar CORS para que o back aceitasse requests do localhost:3000



agora listando a fila no painel principal



agora, exibir:

do pdf: "Para cada motorista: nome, placa, produto solicitado, horário de chegada e tempo de espera em minutos"

percebi que eu não fiz os JOINS da query para poder buscar infos do motorista e produto na hora de retornar a fila

ANTES DISSO AINDA: percebi que precisaria melhorar a questão de criar/atualizar dados do motorista na hora de criar entrada

Agora a função checa se

- 1- o motorista com o cpf, cnh ou placa informados já existe
 - 2- se existe, verifica se todos esses outros dados que são UNIQUE estão correspondentes
 - 3- se estiver tudo ok, agora a única coisa que pode ser atualizada é o nome
- antes dava update em tudo no caso de conflito (exceto cpf, mas está errado pq CNH é identidade e, NESSE CENÁRIO, a placa também, já que é usada pra identificar motorista)

The screenshot shows a development environment with three main panels:

- REST Client (Top Left):** A curl command is shown for a POST request to `http://localhost:8080/api/v1/fila/` with a JSON body:


```
{
    "nome": "ciclano da silva",
    "cpf": "10987654321",
    "cnh": "987612345",
    "placa": "BCA1243",
    "produto_id": 2
  }
```
- Database Table (Top Right):** A table named 'entradas_fila' is displayed with columns: id, motorista_id, produto_id, status, horario_chegada, inicio_carregamento, and fim_carregamento. It contains 6 rows of data.

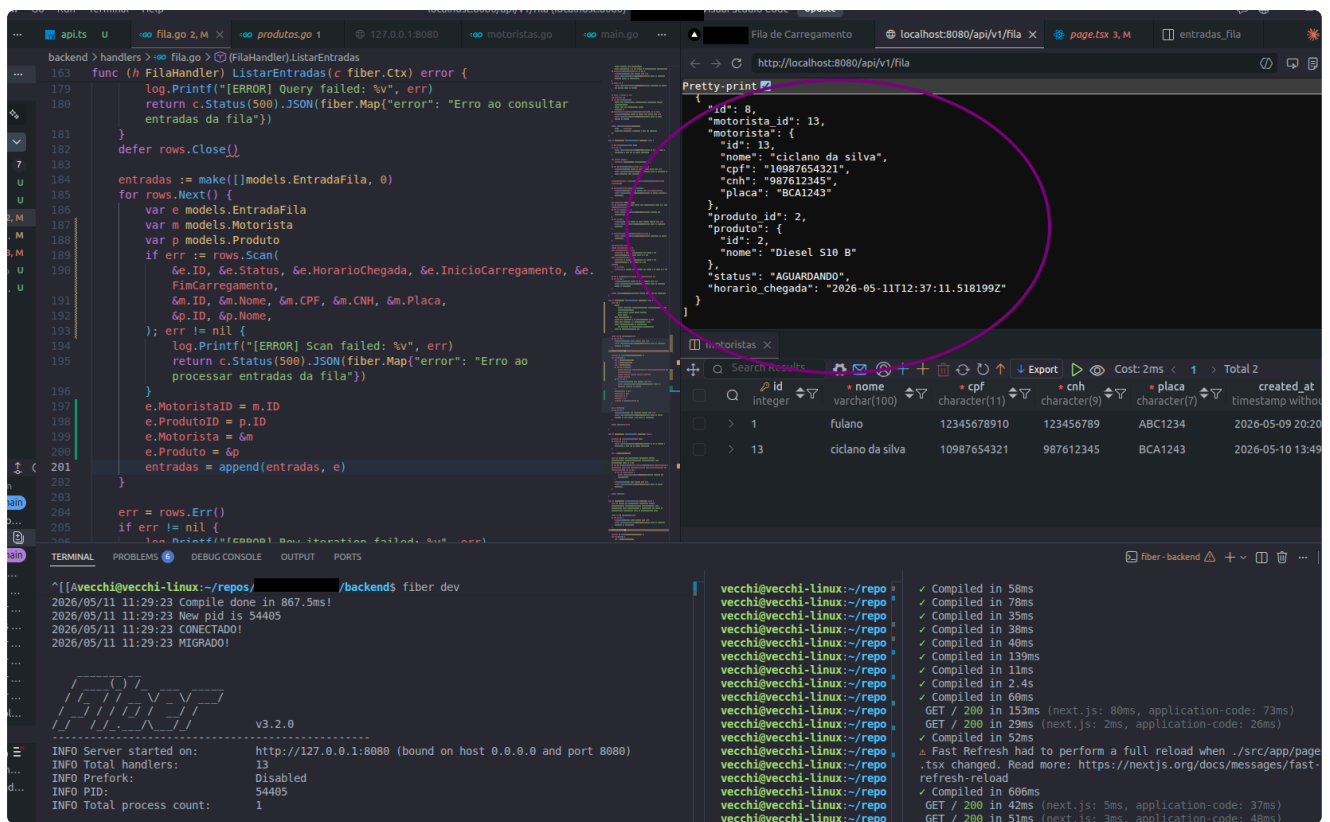
id	motorista_id	produto_id	status	horario_chegada	inicio_carregamento	fim_carregamento
6	13	2	CANCELADO	2026-05-09 13:52:37.46606	(NULL)	(NULL)
3	1	1	FINALIZADO	2026-05-10 04:07:50.41260	2026-05-10 20:11:10.07768	2026-05-10 20:11:10.07768
7	13	2	FINALIZADO	2026-05-11 13:52:37.46606	(NULL)	(NULL)
5	13	2	CANCELADO	2026-05-11 13:52:37.46606	(NULL)	(NULL)
8	13	2	AGUARDANDO	2026-05-11 12:37:11.51815	(NULL)	(NULL)
- Terminal (Bottom):**
 - Left Panel:** Shows logs for the 'fibers' process, including server startup, PID changes, and compilation times.
 - Middle Panel:** Contains a JavaScript function `verificarDivergenciaNosDadosFornecidos` that checks for unique constraints (CPF, CNH, PLACA) in the database. It uses a REST client to query the API.


```
function verificarDivergenciaNosDadosFornecidos (CPF, CNH ou placa) {
  const url = `http://localhost:8080/api/v1/fila/?cpf=${CPF}&cnh=${CNH}&placa=${placa}&produto_id=2`;
  const response = await fetch(url);
  if (response.status === 200) {
    // Motorista já possui uma entrada ativa na fila
    return { error: true, message: "Motorista já possui uma entrada ativa na fila" };
  } else {
    // Motorista não possui uma entrada ativa na fila
    return { error: false, message: "Motorista adicionado à fila com sucesso" };
  }
}
```
 - Right Panel:** Shows the output of the REST client, including the JSON response and the status of the request (e.g., 200 OK).

agora preparar o JOINS no backend

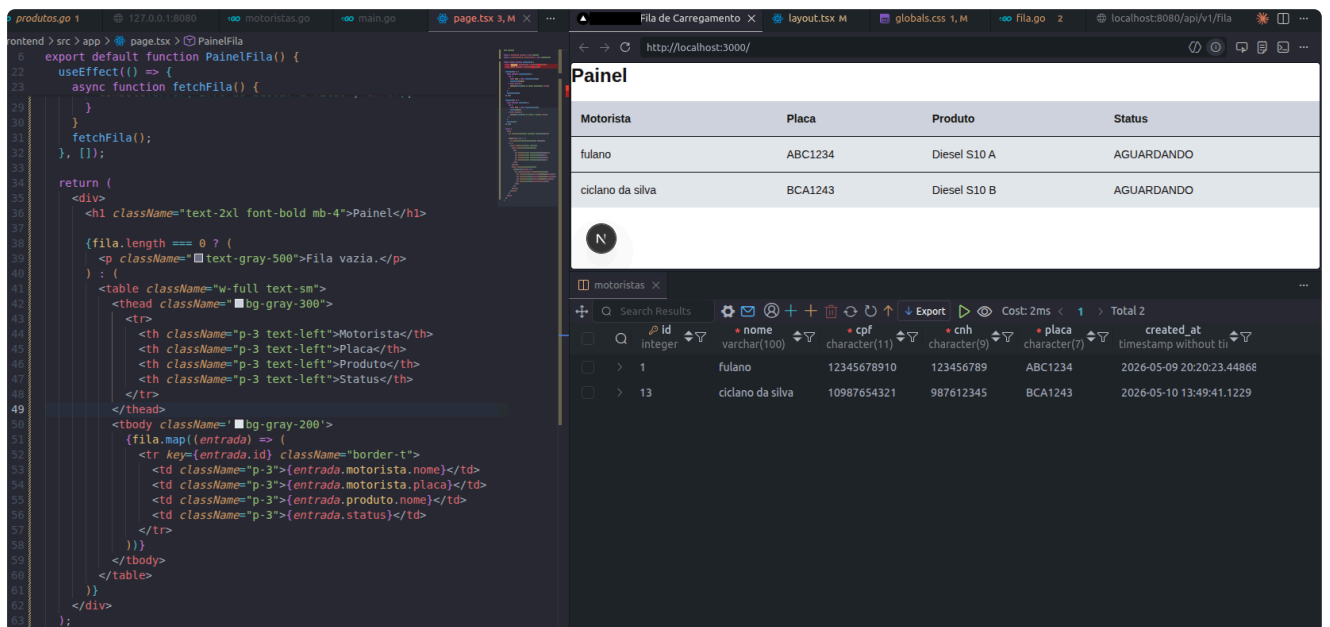
depois -> front: "Para cada motorista: nome, placa, produto solicitado, horário de chegada e tempo de espera em minutos"

com os JOINS e atualizando o Scan() para montar o EntradaFila completo (com motorista e produto) fica assim



exibindo a fila no painel em uma tabela

vou melhorar a exibição da fila -> trocar o JSON cru por uma tabelinha pelo menos



(12/05)

PAUSE PARA PREPARAR A ENTREGA (docker)

como já estou no limite do prazo de entrega, vou criar um docker-compose e preparar tudo para enviar o projeto, daí continuo após isso

- ✓ usar variáveis env no db.go para configurar o banco e em main.go (url do front)

```
vecchi@vecchi-linux:~/repos, [redacted] $ go version
go version go1.26.2 linux/amd64
```

✓ Dockerfile

<https://docs.docker.com/build/building/best-practices/>

<https://hub.docker.com/search?badges=official>

https://hub.docker.com/_/postgres

https://www.reddit.com/r/selfhosted/comments/1nsjxm2/if_youre_moving_to_docker_postgres_18_you_should/

<https://docs.docker.com/compose/gettingstarted/>

✓ docker-compose.yml

✓ .dockerignore → peguei exemplos para Go e postgresql

... fiz a orquestração das inicializações dependendo do makefile e só percebi agora que no windows não é nativo

vou transformar em package.json na raiz para rodar com npm

tela de criar entradas

preciso urgente da tela de criar entradas senão o frontend não serve pra nada

o necessário é um form pra fazer o POST com os dados necessários

estou tendo muita dor de cabeça tentando passar a carroça na frente dos bois e não lidar com a tipagem ANTES de fazer as coisas

```
export default function EntradasPage() {
5   <h1 className="text-xl font-bold mb-4">Entrada na Fila</h1>
6
7   <form onSubmit={handleSubmit} className="space-y-3">
8     <input required value={nome} onChange={e => setNome(e.target.value)}
9       placeholder="Nome" className="w-full border p-2 rounded" />
10    <input required value={cpf} onChange={e => setCpf(e.target.value)}
11      placeholder="CPF" className="w-full border p-2 rounded" />
12    <input required value={cnh} onChange={e => setCnh(e.target.value)}
13      placeholder="CNH" className="w-full border p-2 rounded" />
14    <input required value={placa} onChange={e => setPlaca(e.target.value.toUpperCase())}
15      placeholder="Placa" className="w-full border p-2 rounded" />
16
17    <select required value={produtoId}
18      className="w-full border p-2 rounded">
19      <option value={0} disabled>Selecione um produto
20      {produtos.map(p => (
21        <option key={p.id} value={p.id}>{p.nome}</option>
22      ))}
23    </select>
24
25    {erro && <p className="text-red-500 text-sm">{erro}</p>}
26
27    <button type="submit" className="w-full bg-blue-600 text-white p-2 rounded">
28      Confirmar Entrada
29    </button>
30  </form>
31 </div>
32 )
33 }
34 }
```

vou tirar um tempopara lidar com isso direito logo (é que antes estava fazendo testes rápidos para ver se conseguia conectar o front ao back, mas agora que já testei, está na hora de definir types já....)

fiz a tipagem (trazer os types que defini em models.go)

pra não ter que ficar repetindo a mesma estrutura de fetch toda vez, criei um helper/wrapper ("fetcher")

```
6 // concentra as chamadas em um helper para deixar mais limpo e tratar
  de erros em um lugar só
7 async function fetcher<T>(endpoint: string, options?: RequestInit):
  Promise<T> {
8   const response = await fetch(`${BASE_URL}${endpoint}`, {
9     headers: {
10       'Content-Type': 'application/json',
11     },
12     ...options,
13   });
14
15   const jsonData = await response.json();
16
17   if (!response.ok) {
18     // o backend sempre manda {error: ...}, então tento usar isso,
19     // mas se não der -> mensagem genérica de erro http
20     throw new Error(jsonData.error ?? `HTTP ERROR - status: ${response.
21       status}`);
22   }
23
24   return jsonData as T;
25 }
```

pq assim as outras funções ficam bem mais limpas:

```
6 export async function listarProdutos(): Promise<Produto[]>{
7   return fetcher('/produtos')
8 }
9
10 export async function listarEntradas(): Promise<EntradaFila[]> {
11   return fetcher('/fila');
12 }
```

Agora fiz a pagina com o formulario para criar entradas (+ botão na pagina principal para levar para o form) - JA FUNCIONA

localhost:3000

Painel

+ Nova Entrada

Motorista	Placa	Produto	Status
TESTE FRONTEND	XYZ2026	Gasolina A	AGUARDANDO
ALEXANDRAO	GAZ4144	Diesel S10 A	AGUARDANDO
AAAA	AAAAAAA	Gasolina C	AGUARDANDO

Entrada na Fila

Selecione o produto...

▼

Criar Entrada

agora dá pra criar entradas e ver a fila, mas não dá pra fazer nada com elas
Falta adicionar as ações no painel que vão rodar a atualização de status

atualizar status pelo painel

agora podemos avançar (aguardando->carregando->finalizar) ou cancelar cada entrada

```
{fila.map((entrada) => (  
  <td className="p-3">{entrada.motorista.placa}</td>  
  <td className="p-3">{entrada.produto.nome}</td>  
  <td className="p-3">{entrada.status}</td>  
  <td className="p-3 flex gap-2">  
    {entrada.status === 'AGUARDANDO' && (  
      <button  
        onClick={() => handleAvancar(entrada.id,  
          'CARREGANDO')}  
        className="bg-blue-500 text-white p-2 text-xs"&br/>      >  
        Iniciar  
      </button>  
    )}  
    {entrada.status === 'CARREGANDO' && (  
      <button  
        onClick={() => handleAvancar(entrada.id,  
          'FINALIZADO')}  
        className="bg-green-500 text-white p-2 text-xs"&br/>      >  
        Finalizar  
      </button>  
    )}  
  </td>  
)}
```

Painel

+ Nova Entrada

Motorista	Placa	Produto	Status	Ações
g	GGGGGGG	Diesel S10 A	CARREGANDO	Finalizar Cancelar
aaa	AAA5555	Diesel S10 B	AGUARDANDO	Iniciar Cancelar

(13/05)

para deixar o app com todas as funcionalidades, pelo menos, falta o historico

uma consideração sobre o painel e "fila do dia"

ANTES, UMA CONSIDERAÇÃO:

abri o painel e não vi as entradas de ontem

percebi que é porque a fila só mostra entradas do dia

PORÉM como ficam as entradas que ainda estão AGUARDANDO/CARREGANDO?

não faz sentido ficarem no historico porque não estão finalizadas, mas também não seria um bom design fazer com que elas sumissem da fila e ficassem basicamente inacessíveis...

Então, apesar das instruções do pdf do projeto mencionarem `GET /fila -> "Lista todos os motoristas na fila do dia, ordenados por chegada"`, vou tirar esse trecho da query

```
func (h FilaHandler) ListarEntradas(c fiber.Ctx) error {  
  query := `  
    SELECT  
      ef.id, ef.status, ef.horario_chegada, ef.inicio_carregamento, ef.fim_carregamento,  
      m.id, m.nome, m.cpf, m.cnh, m.placa,  
      p.id, p.nome  
    FROM entradas fila ef  
    INNER JOIN motoristas m ON ef.motorista_id = m.id  
    INNER JOIN produtos p ON ef.produto_id = p.id  
    WHERE ef.horario_chegada >= CURRENT_DATE  
    AND ef.status IN ('AGUARDANDO', 'CARREGANDO')  
    ORDER BY ef.horario_chegada ASC  
  `
```

- VAI QUE alguma entrada foi largada para trás -> operador precisa ficar ciente para lidar com ela e ela poder aparecer no historico
- VAI QUE a base de carregamento funciona ao longo da noite/madrugada -> passando da meia noite, atrapalharia todo o sistema pq todas as entradas criadas antes disso iriam desaparecer

tela do histórico

voltando: para deixar o app com todas as funcionalidades, pelo menos, falta o historico

a função de listar histórico lá no handler estava ainda sem os JOINS de motorista e produto. Em vez de copiar tudo de novo que usei no ListarEntradas, já refatorei e deixei a parte comum da query como uma constante e também extraí a função de Scan() (que era enorme) para reutilizar

pagina do historico agora funciona e exibe as entradas encerradas

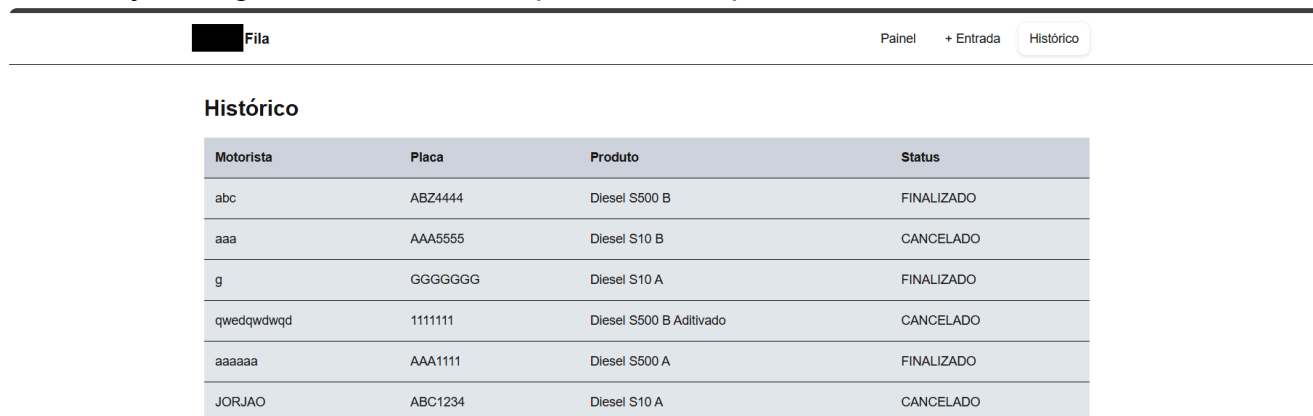


navbar

já está incômodo ficar editando a url toda vez para voltar para o painel principal. Vou implementar uma navbar logo pra facilitar a navegação

(14/05)

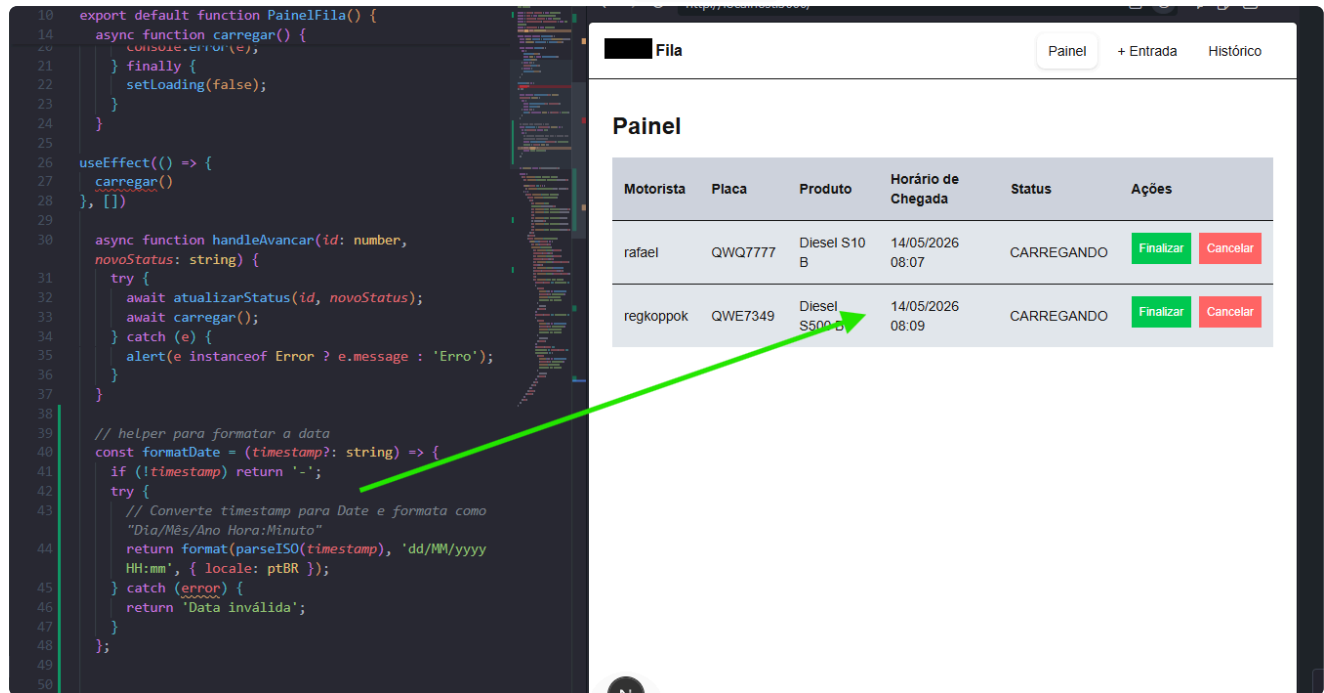
também já fiz algumas melhorias simples de estilo para ficar *menos horroroso*



painel -> horario de chegada + tempo de espera

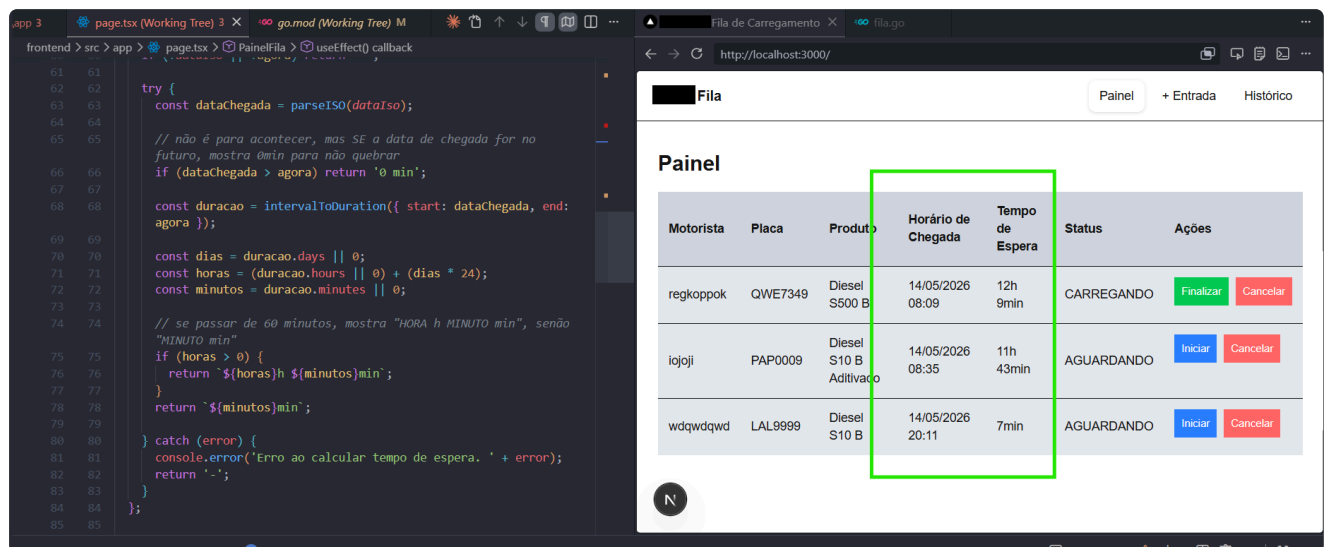
vou finalmente exibir o horário de chegada de cada motorista (preciso ver como formatar, por isso tinha deixado para depois)

agora exibe o horário já formatado (usando `date-fns`)



para calcular o tempo de espera também posso usar o `date-fns` ("intervalToDuration")

feito:



agora as badges de status de acordo com especificações do projeto
não ficou a melhor coisa do mundo esse carregando, mas ok:

Fila

Painel+ EntradaHistórico

Painel

Motorista	Placa	Produto	Horário de Chegada	Tempo de Espera	Status	Ações
regkoppok	QWE7349	Diesel S500 B	14/05/2026 08:09	12h 33min	Carregando	Finalizar Cancelar
iojoji	PAP0009	Diesel S10 B Aditivado	14/05/2026 08:35	12h 6min	Aguardando	Iniciar Cancelar
wdqwdqwd	LAL9999	Diesel S10 B	14/05/2026 20:11	31min	Aguardando	Iniciar Cancelar

Fila

Painel+ EntradaHistórico

Histórico

Motorista	Placa	Produto	Status
abc	ABZ4444	Diesel S500 B	Finalizado
aaa	AAA5555	Diesel S10 B	Cancelado
g	GGGGGGG	Diesel S10 A	Finalizado
qwedqwdwqd	1111111	Diesel S500 B Aditivado	Cancelado
aaaaaa	AAA1111	Diesel S500 A	Finalizado
JORJAO	ABC1234	Diesel S10 A	Cancelado

agora é o contador de motorista em cada status

15/05

estou reaproveitando o mapeamento status->estilo e status->label do StatusBadge no StatusCounter agora

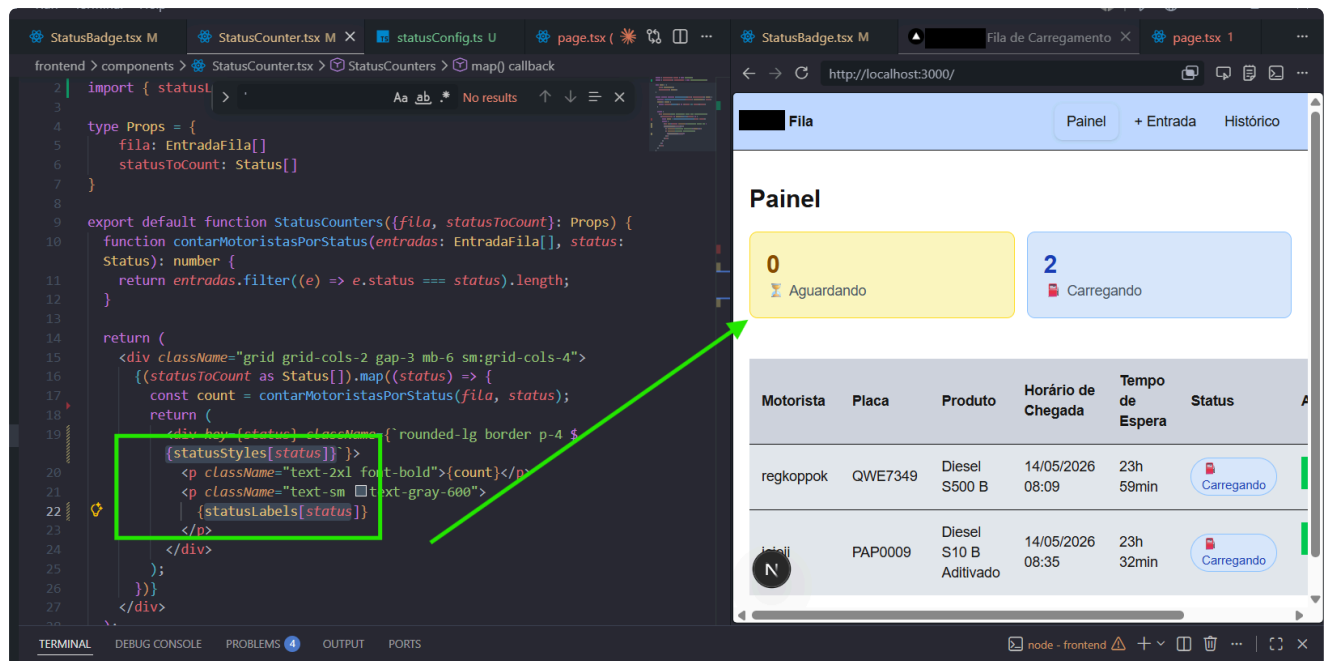
<https://refine.dev/blog/typescript-record-type/#what-is-record-type-in-typescript>

e descobri o type Record (tinha deixado vazio aqui porque não sabia o que usava para um

cenário assim)

```
6 FINALIZADO: "bg-green-100 text-green-800 border-green-300",
7 CANCELADO: "bg-red-100 text-red-800 border-red-300",
8 };
9
10 export const statusLabels: Record<Status, string> = {
11   AGUARDANDO: "⌚ Aguardando",
12   CARREGANDO: "🚚 Carregando",
13   FINALIZADO: "✅ Finalizado",
14   CANCELADO: "❌ Cancelado",
15 };
```

type Record<K extends string | number | symbol, T> = { [P in K]: T; }
Construct a type with a set of properties K of type T



5.3 Tela 3 — Histórico

Tela que lista os carregamentos finalizados e cancelados. Deve exibir:

- Filtro por data
- Para cada entrada: motorista, produto, horário de chegada, horário de finalização e tempo total na base



6. Regras e restrições técnicas

isso vai dar pra calcular para o FINALIZADO.... mas CANCELADO não armazenamos horário de finalização (só fim_carregamento)

teria que consertar isso na modelagem, mas PRA AGORA não vai ter tempo